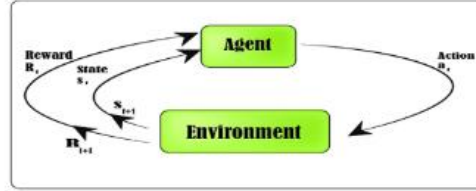


I. DEFINE AN ENVIRONMENT AND THE PROBLEM TO BE SOLVED (TASK1)

BASIC TASK

Figure 1RL algorithm process by *Chakole et al.* [1]



My environment is the stock market trading, and my agent behaves as a stock trader. Basically, there are 4 statements and the agent choose one action out of 3 actions (do nothing/hold, buy or sell). The agent obtains the state information from the data. The state depends on the stock price change information in every day level with chronological order and the number of current stock holds (they are explained more in later section). In sum, my agent meets one state out of 4 states and, in each state my agent chooses one action out of 3 actions. (Actually, in some states, agent cannot take a part of actions, but it is explained in the later section.)

When the agent chooses the action, there are mainly 4 constraints. One is the budget constraint. This constraint prohibits the agent from having negative budget amount. Intuitively, when the trader buys some stocks, it loses some money from its budget. Thus, if the trader has no money, it cannot buy any stocks. In my environment, if the budget is zero or negative, the agent exits the environment as an early stopping point. Another constraint is that the agent cannot have the negative amount of stock holding. Again intuitively, if the trader has no stocks, it cannot sell any stocks. In real stock market, the trader can sell the stocks, even though it cannot actually have the stock itself, which is so called short selling. However, to avoid the complexity of my environment, short selling is prohibited. In my environment, if the agent has the negative stock amount, then it exits the environment as another early stopping point. The third constraint is that my agent is only able to have 0 or 1 stock amount. This is simply because I would like to simplify the stock trading environment and have the Q value convergence. The final constraint is about the initial amount of stock holding. In my environment, my agent has 0 or 1 stock randomly as the initial amount of stock holding. This is because the way of making states is related to the amount of stock holding and thus, without this randomness, the initial state cannot be chosen randomly in every episode. In addition, this random initial condition allows the agent to explore my environment (the way of making states will be explained in section II).

The goal of my problem is for the agent to learn the optimal stock trading policy. More precisely, my goal is that my agent finds the optimal stock trading strategy from the stock market movement data. Thus, in my problem, Q table is interpreted as the trading strategy, and the convergent of Q table indicates that my agent finds the optimal trading strategy. To achieve learning optimal stock trading strategy, my agent tries to maximize the cumulative reward during the given period of data.

In my environment, I use the stock price related data. By using the real-world data, the agent can learn the optimal stock trading strategy. I extracted the data from [2]. The data is APPL (APPLE INC stock data) from 1998 to 2018. It includes date, open price, high price, low price, close price and trading volume every day. For my task, I use the last 300 data as the stock market information. Thus, the time steps in each episode are 300. To see the convergence of Q value, 2000 episodes are executed. The same data is applied to task 7 and 8 (Advanced Task).

II. DEFINE A STATE TRANSITION FUNCTION AND THE REWARD FUNCTION(TASK2)

A. Action Space

As I mentioned in the former section, my agent takes only 3 actions; hold/do nothing, sell or buy. More precisely, hold or do nothing indicates that if the agent has one stock, then it is called “hold”, and if the agent does not have stocks, then it is called “do nothing”. As for “sell” and “buy”, these mean that my agent sells one stock for “sell” and buy one stock for “buy”. The reason is why I specify the stock amount as one is that I define such an environment (agent can only have 0 or 1 stock). The representative of these 3 actions as follows:

Hold/do nothing	Buy	Sell
[1,0,0]	[0,1,0,]	[0,0,1]

B. State Space and State Transition Function

In this section, I will explain how I made the state and show the state transition function, and state transition probability. If I use daily price change data directly, the states are infinitely many (continuous States). This breaks one MDP definition, a finite set of states, S . Therefore, to discretize the states, I use 2 information as follows:

- The daily change of stock price: (Close t – Open t). This comes from data (t means the time at t)
- The amount of current stock holding: 0 or 1. This comes from how many stocks the agent has currently.

So, states can be described as follows: State = [The daily change of stock price, the amount of current stock holding]

Based on this 2 information, the states are constructed. The states are partitioning clustered by 2 criteria: the daily change of stock price is more or less than 0 and the current stock holding amount is more than 1 or less than 0. Then, by using these criteria, I successfully split the states into only 4 states. The reason why I make the states in this way is that I try to imitate the real-world condition. If the stock trader has some stocks and the price change becomes positive, then the trader will sell the stocks or hold them to expect the stock price to keep increasing. However, if the trader has no stock and the price change becomes

positive, then the trader tends to do nothing or buy some stocks. These real-world stories tell me that the state depends on the current stock price and also the amount of current stock holdings. Thus, in my environment, the states are made by this 2 information, and then I obtain 4 states by the partitioning clustering. Therefore, the states in my environment can be repeated. Now, by discretizing the states, my trading environment can be described as MDP.

In Markov decision process, a state moves onto the next state with a probability $P(S_{t+1} | S_t, A_t)$ called State transition probability. Thus, state transition function is defined as follows:

$$S_{t+1} = \delta(S_t, a_t) \quad (1)$$

In my environment, because of the construction of my states (using stock price change data and current stock holding information), the next state depends on the past state and also past action (the amount of holding stock is interpreted as the result of the action). As for the terminal state, my agent has no specific terminal state; instead, the agent observes the stock price change every day chronologically until the end of the data; thus, I would say that the terminal state is equivalent to the last state in the data in my environment.

For state transition probability, I assume that it is initially uniformly distributed, and it will be updated. the way of updating state transition matrix is that I just count the movements from one state to another state. Figure 2 shows the state transition matrix after the training of the base experiment (the experiment in section IV). It is well distributed.

Figure 2 Transition probability matrix

	s0	s1	s2	s3
s0	0.000140	0.002602	0.000152	0.001269
s1	0.280103	0.005622	0.222054	0.004182
s2	0.015439	0.018250	0.012342	0.014315
s3	0.233178	0.005660	0.180333	0.004359

C. Reward function

For reward values, they are constructed based on the stock price data. I describe 4 discretized states from the many states. Thus, within the same states, I should take an average of the stock prices to construct the instant rewards. In addition, I add some constrains corresponding to the real-world trading situation in some instant reward values. In state s0 (positive reward and stock holds) and s2(negative reward and stock holds), the agent cannot take action “buy”, because I only allow the agent to have up to 1 stock amount. Thus, the rewards at these positions are nan. In state s1, (positive reward and No stock holds) and s3 (negative reward and No stock holds), the agent cannot take an action “sell” because I do not allow the agent to have negative stock amount. Thus, the rewards at these positions are nan. In addition, in state s0 and s2 (both in stock holds), I add holding reward for s0 (positive reward) and penalty for s2 (negative reward). Whole instant reward as I mentioned above is described in the Figure 3. In figure 3, index and column’ order are as follows:

Index= [s0,s1,s2,s3] (in the graph, from Top to the bottom)

Column = [a0 = hold/do nothing, a1 =buy, a2 =sell] (in the graph, from left to right)

Thus, for example, $R[s0, a0 = \text{hold/do nothing}] = 0.25$, $R[s3, a2 = \text{sell}] = \text{nan}$ in figure 3.

Figure 3 Reward Table

[	0.25	nan	2.5	]
[	0.	0.	nan]	
[-0.28	nan	-2.8	]	
[	0.	0.	nan]	

III. SET UP Q LEARNING PARAMETERS (GAMMA, ALPHA) AND POLICY(TASK3)

A. Q-learning parameters

In my environment, all Q values has initial values as 0s, and the agent updates the Q matrix values to find the optimal policy in order to maximize the cumulative reward. The Q value update function is as follows:

$$Q_{old}(s,a) = Q_{old}(s,a) + \alpha * [r(s,a) + \gamma * \text{argmax}_{a'} Q(s',a) - Q_{old}(s,a)] \quad (2)$$

The most left hand side of $Q_{old}(s,a)$ represents the updated old q-value, other $Q_{old}(s,a)$ represents current Q value, $r(s,a)$ is the instant reward and $\text{argmax}(s',a)$ is the choice of max Q value in the next state.

In this equation, there are 2 hyper parameters α and γ to have the impact on the algorithm performance.

- α is a learning rate that has the impact on the learning step size[3]. In other words, it represents how fast the new q value should be modify from the old q value. α is between 0 and 1. A with high value such as 0.9 indicates that an algorithm learns very fast. On the other hand, $\alpha = 0$ indicates that Q value is not updated.
- As for γ , it is the discount rate. It tells us how much the agent cares about the future reward. $\gamma = 0$ means that the agent only cares about the immediate current reward, while $\gamma = 1$ means that the agent only care about the future reward.

In my environment, stock trading environment, I set $\alpha = 0.5$ and $\gamma = 0.5$ for the base experiment in section IV. As for α value choice, just I would like my agent to learn the environment moderately fast. As for γ value choice, I would like my agent to weight the immediate (current) reward and the future reward equally to successfully maximize the cumulative reward. This is because the trader who too much cares about the immediate reward is likely to lose their money. The balance is important in stock trading market.

B. Policy

In my environment, the current agent action is selected based on ϵ -greedy policy. In this policy, the agent is allowed to explore the environment in the earlier episodes, and then the agent tends to take greedy action (exploit) by reducing ϵ value after the end of every episode. In this policy, the agent takes its action randomly with the probability of ϵ , while it takes greedy action with the probability $1 - \epsilon$. The purpose of the policy is that the agent finds the optimal policy in order to maximize the cumulative reward. The policy is described as follows:

$$\pi(st) = at \quad (3)$$

The reason why I choose this policy is that just I am familiar with this policy compared with other policies such as random policy and greedy policy.

Normally, ϵ value starts at very high value such as 0.9 and then until it becomes 0.5, this value is multiplied by 0.99999 after each episode; then, after ϵ value becomes less than 0.5, it is multiplied by 0.9999 [3]. Thus, I basically follow this criterion but modify a bit. I set initial ϵ value as 0.9 and decay rate as 0.9999 if $\epsilon \geq 0.5$, and decay rate as 0.999 if $\epsilon < 0.5$. Then, ϵ will be reduced until $\epsilon = 0.01$.

IV. RUN THE Q-LEARNING ALGORITHM AND REPRESENT ITS PERFORMANCE(TASK4)

A. Q learning algorithm in my environment

In my environment, all Q values has initial values as 0s, and the agent updates the Q matrix values to find the optimal policy in order to maximize the cumulative reward. The Q value update function is followed by equation (2). The most left hand side of $Q_{old}(s,a)$ represents the updated old q-value, other $Q_{old}(s,a)$ represents current Q value, $r(s,a)$ is the instant reward and $\arg\max(s',a)$ is the choice of max Q value in the next state.

In this base experiment, as I mentioned in the prior sections, I set $\alpha = 0.5$, $\gamma = 0.5$ for 2 hyper parameters. In addition, about the policy, ϵ value starts at 0.9, and it is reduced by multiplied by 0.9999 until ϵ value = 0.5; then it decreases again by multiplied by 0.999 until ϵ value = 0.01 [3].

Figure 4 Q matrix after training

$$\begin{bmatrix} 0.89158526 & 0. & 3.12537052 \\ 1.34647625 & 0.62479325 & 0. \\ 1.22780989 & 0. & -2.20303526 \\ 1.13421249 & 1.09821133 & 0. \end{bmatrix}$$

The figure 4 shows the Q matrix after the training is end. In the process of learning the optimal policy is as follows (show one sample by using above Q table):

- Agent meets the state 0 (s_0)
- Agent has the environment information based on the Q matrix (a_0 = hold/do nothing, a_1 = buy, a_2 = sell) below

$$Q(s_0, a_0) = 0.891, Q(s_0, a_1) = 0, Q(s_0, a_2) = 3.125$$

Then with probability ϵ , the agent chooses any one out of three actions randomly, while with probability $1 - \epsilon$, the agent selects a_2 , because $Q(s_0, a_2)$ has the highest value. Then, the agent obtains the instant reward r corresponding to $R(s_0, a_2)$ (see figure 3), then, Q value ($Q(s_0, a_0)$, $Q(s_0, a_1)$, or $Q(s_0, a_3)$) is updated based on eq (2). The agent repeats this process. In each episode, the agent learns the environment basically during 300 steps (if budget < 0 or stock holds ≤ -1 , then the agent exits the environment and goes to the next episode); and then it repeats 2000 episodes to have the Q-matrix value convergence.

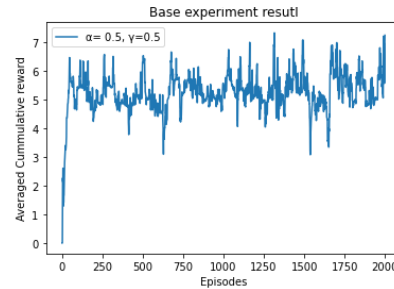
B. Represent the performance

Again, figure 5 shows the Q table after the training and figure 6 shows the average of Q table values in the last state in each episode. Figure 6 represents whether Q values converges or not in my environment. Interpretation of figure 6, all Q values almost converges, but not perfectly converges.

Figure 5 Q matrix of the base experiment in section IV

$$\begin{bmatrix} 0.89158526 & 0. & 3.12537052 \\ 1.34647625 & 0.62479325 & 0. \\ 1.22780989 & 0. & -2.20303526 \\ 1.13421249 & 1.09821133 & 0. \end{bmatrix}$$

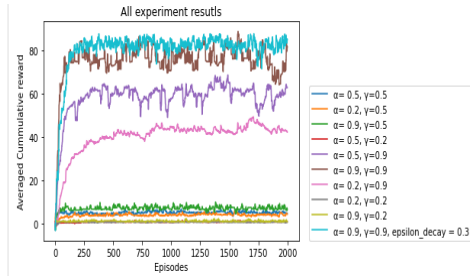
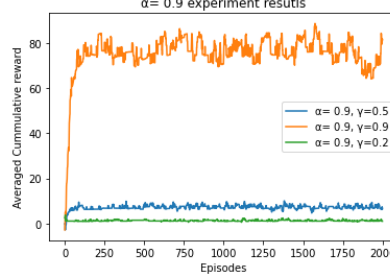
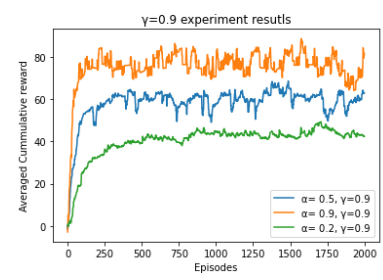
Figure 6 Averaged cumulative reward through episode



V. REPEAT THE EXPERIMENT WITH DIFFERENT PARAMETER VALUES AND POLICES(TASK5)

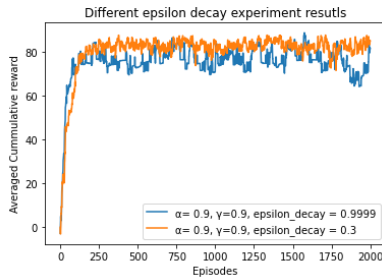
After the base experiment in section IV, I run the algorithm with different hyper parameters. The hyper parameters are combinations of α , γ and decay rate of ϵ . The options of parameters are as follows: $\alpha = [0.2, 0.5, 0.9]$, $\gamma = [0.2, 0.5, 0.9]$, $\epsilon_{decay} = [0.9999, 0.3]$. First of all, I change α and γ , but keep $\epsilon_{decay} = 0.9999$. After specifying the best experiment (best model), then for the best model, I will change epsilon decay to see how epsilon decay works in my environment. Thus, I totally 10 experiments. The figure 7 represents the all cumulative rewards for 10 experiments. Figure 7 of course includes the base experiment in section IV.

Figure 7 10-experiment averaged cumulative reward

Figure 8 Experiments with same $\alpha=0.9$ Figure 9 Experiments with the same $\gamma=0.9$ 

As for the figure 8, it shows 3 experiments with the same $\alpha=0.9$. In my experiment, I realized α value is not so important (see the analysis section in details), and so I put one graph for the same α values. In figure 9, I will analyze $\gamma=0.9$ experiments, and so I put it here. Overall, with the same γ , higher α leads to the higher averaged cumulative reward. Figure 10 shows that the best performed model with different epsilon decay rate 0.3 or 0.9999. This comparison shows how the epsilon decay rate has the impact on the cumulative reward. Overall, there is no huge impact on the cumulative reward.

Figure 10 Best model comparison with different epsilon decay

Figure 11 Q matrix $\alpha=0.5, \gamma=0.5$

$\begin{bmatrix} 0.89158526 & 0. & 3.12537052 \\ 1.34647825 & 0.62479325 & 0. \\ 1.22780989 & 0. & -2.20303526 \\ 1.13421249 & 1.09821133 & 0. \end{bmatrix}$

Figure 12 Q matrix $\alpha=0.5, \gamma=0.9$

$\begin{bmatrix} 8.60337941 & 0. & 9.9194094 \\ 8.15685637 & 5.74960847 & 0. \\ 6.20199157 & 0. & 4.69518843 \\ 8.19863103 & 5.83091353 & 0. \end{bmatrix}$

Figure 11 and 12 describe the Q matrix table of $\alpha=0.5, \gamma=0.5$ and $\alpha=0.5, \gamma=0.9$ experiments (epsilon decay is 0.9999). These graphs represent how γ is important factor for the cumulative reward and also how optimal strategy becomes different based on γ in my environment (analyze details in section VI).

VI. ANALYZE THE RESULTS QUANTITATIVELY AND QUALITATIVELY(TASK6)

The Q matrix comparison is explained. In the base experiment in section 4, I ran it with $\alpha=0.5, \gamma=0.5, \epsilon$ decay = 0.9999. In another experiment in section 5, I ran it with $\alpha=0.5, \gamma=0.9, \epsilon$ decay = 0.9999. I will interpret the Q value matrix of them as the representatives in the stock market environment context.

As the figure 11 (Q matrix of $\alpha=0.5, \gamma=0.5$ case), In state s0 (PROFIT>0 & stock holds), selling is the best action to have the biggest reward. In state s1(Profit >0 & no stock), do nothing is the best action. In state 2 (Profit < 0 & stock holds), do nothing (hold) is the best action. In state s3 (Profit < 0 & No stock), do nothing is the best action, but buying is also good action. These are because these actions clearly have the high Q values within each state.

As for $\alpha=0.5, \gamma=0.9, \epsilon$ decay = 0.9999 model (see figure 12), in state s0 (PROFIT>0 & stock holds), selling and holding have the almost equal important, because selling and holding has near Q values. In state s1(Profit >0 & no stock), do nothing is the best action. In state 2 (Profit < 0 & stock holds), do nothing (hold) is the best action clearly because it has the biggest Q value. In state s3 (Profit < 0 & No stock), do nothing is the better action based on the correspond Q value, but buying is also a good action because of the high Q value.

Between these experiments, State s2 is clearly different based on the Q values. In the base experiment (see figure 11), $\gamma=0.5$ suggests that the recent reward and future reward are equally important. Thus, agent tends to hold the stock for shorter period, and in my environment, it means that the agent does not care about holding reward. So action, do nothing, in state s2 has the negative value (-2.20)(see figure 11). On the other hand, in the other experiment, $\gamma=0.9$ (see figure 12) indicates that future reward is huge weighed. This implies that the agent is likely to hold the stock for a long time because the agent loves getting the holding reward. Thus, the Q-value of state s3 & action, hold, has the positive reward (+4.69) (see figure 12).

In the contest of the stock trading market, these results show that the trader should weight the future reward, and so basically holding the stock for a long time is a good strategy, even though the stock price decreases for a short term.

Next, I interpret the cumulative reward graphs. The figure 7 shows that overall, the higher gamma ($\gamma=0.9$) experiments are likely to have the higher cumulative rewards. This suggests that as I mentioned above, the strategy of weighting the future reward leads to earning the higher reward.

Then, I will explain the stability of q values in the long run by using figure 9. In $\alpha=0.9, \gamma=0.9$ experiment and $\alpha=0.5, \gamma=0.9$ experiment, these experiments have the higher cumulative rewards, but the convergence is relatively low. On the other hand, $\alpha=0.2, \gamma=0.9$ experiment has high cumulative reward, but the Q value convergence is more stable than the above 2 experiments. This suggest that late learning α makes the trader earn money in a relatively stable manner.

Finally, I will analyze how epsilon decay level has impact on the algorithm performance in the best performed model ($\alpha=0.9, \gamma=0.9$ model). The figure 10 shows that both of them lead Q value to converge very fast. Thus, different epsilon decay rate has no big impact on the amount of Q values in my stock trading environment. However, low epsilon decay = 0.3 makes Q value convergence more stable than high epsilon decay = 0.9999 (see figure 10). Thus, I would say that epsilon decay has some what impact on the Q value stability as well as α in my stock market environment.

ADVANCED TASK

VII. IMPLEMENT DQN WITH 2 IMPROVEMENTS AND MOTIVATION OF MY CHOICE AND EXPECTATIONS FOR CHOOSING A PARTICULAR IMPROVEMENT IN THE SELECTED ENVIRONMENT(TASK7)

A. Chose environment, Stock trading market

I select the environment stock trading market as I developed in the basic task. However, for this advanced task, I borrow the custom stock market environment from [2] implemented by using Gym, and I modify the environment to run my implemented DQN algorithms. In this environment, agents can choose one action out of 3 actions (sell, buy or hold the stocks). As for the state, this is infinitely many (continuous state space) and basically the information of the daily stock price, but with observing other current market conditions (current stock open price, current stock highest price, current stock lowest price, current stock close price, current stock trading volume, current balance (budget), the number of the current share holds, the cost, total number of share holds, the total value of selling the stocks, current profit). In sum, action and state space in my environment as follows:

- Action = [buy, sell, hold]
- State =[current stock open price, current stock highest price, current stock lowest price, current stock close price, current stock trading volume, current balance (budget), the number of the current share holds, the cost, total number of share holds, the total value of selling the stocks, current profit]

For the current price, it is randomly selected open or close stock price, because I assume that my agent buys, sells or holds with open or close stock price. For simplicity, if the agent chooses buying, it buys the maximum number of stocks within the balance (current budget), if it selects selling the price, then all stock hold amount currently is always sold. If the agent chooses holds (do nothing), then anything does not happen. The agent gets the instant reward with some decay, meaning that if it takes longer days to sell the stocks after buying them, then the reward is discounted. Then, the agent tries to maximize cumulative rewards called the profit in my environment and the performance is evaluated base on the profit.

The initial balance is 10000. The stock price data comes from [2]. This data is the same as the data I use in the basic task. Then, the agent keeps trading stocks up until the specified length (maximum timestep length) and tries to maximize its profit through DQN algorithms. The episode length is 100 and time step in each episode is 1000.

B. Choice of DQN improvements, motivation of the choice and expectations for choosing the particular improvement. Rainbow DQN

First of all, DQN is the extended version of algorithm, consisting of Q-learning and deep neural network to solve complex environments such as infinitely many states and so on. For my environment, the action is discrete and the state space is continuous, and so DQN matches my environment.

My first choice is rainbow DQN, consists of noisy net, n-step, double, dueling and prioritized experience memory (PEM). The reason why I choose Rainbow DQN is that as we learned in the class, it tends to outperform rather than other DQN improvements. This is because I assume that even in very complex environment, stock market trading environment, it works well. The expectations for choosing a particular improvement are as follows:

- Double Q-network: It helps the agent to have the overestimation bias, meaning that if the overestimation bias happens, then the agent tends to choose the action achieving the more reward even if it is just by chance. To avoid this overestimation, one Q-learning is only used for action selection, and the other Q-learning is only used for value action [4].
- PEM: as we know, some experiences have higher values rather than other experiences. In normal experience replay, the positive reward may be very rare, but PEM helps the agent to meet the important states more frequently [4]. As a result, the agent can learn the optimal policy faster than the normal experience replay.
- Dueling: it helps to measure how each action is important by using State value V (the degree of the goodness in one state) and Q value, and it occurs in the neural network components [4]. Thus, Q-network learns the values more efficiently and the agent can have the better performance.
- n-step: In normal DQN, the agent learns by using only 1-step ahead Q-value (next Q-value), while in n-step DQN, then it allows the agent to get the better Q-estimation by using several steps into the future. Then, it appears in the reward calculation and the agent can have the better information from this n-step reward [4].
- Noisy network: This improvement helps to learn exploration strategies. In normal DQN, for the exploration for the action selection, ϵ greedy policy is used. This has several disadvantages: deterministic schedule, limited greediness (slow decay of ϵ value). However, by using Noisy network, this exploration schedule becomes stochastic and the exploration strategies can be optimized for each specific environment or task. (This noise is added in the Q-value estimation and so ϵ -greedy is not necessarily.) [4]

The rainbow DQN is the combinations above improvements; thus, I assume that the agent performs better.

C. Choice of DQN improvements, motivation of the choice and expectations for choosing the particular improvement. Double and Dueling DQN

The reason why I choose only Double and Dueling improvements is how n-step and noisy network has the impact on the agent performance in stock trading market. This is because in stock market trading environment, stock price moves very randomly and also have the time trend, and so I assume that n-step and noisy network work well in my environment. As for the expectation for choosing Double Q network and Dueling architecture, their expectations are explained in section VII B (in rainbow DQN part).

D. Results

Because the agent fails to converge and so, I run 5 experiments for each algorithm (vanilla, dueling and double, and rainbow DQNs) (see figure 14, 15, 16). Then, I take an average of profits in each DQN algorithms. The averaged profits represent the overall performances for each algorithm. In addition, the hyper parameters are kept them the same, because of the failure of the convergence. The fixed important hyper parameters are as follows:

- Gamma = 0.99, memory size = 500, batch size = 64, target update frequency = 200

Also, the episode length keeps 100 and the time step also keeps the same (1000) for all experiments.

Figure 13 just describes all experiment results to see the whole picture of the performances. As for the figure 14, 15, 16, I show 5 experiments in each algorithm to see the convergences. Then The figure 17 shows all averaged profits in all algorithms in one graph to compare the performances with each other.

Figure 13 All experiment results

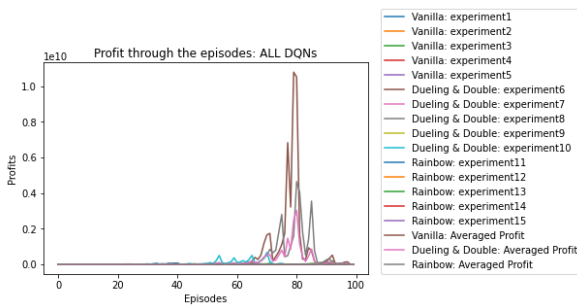


Figure 14 5 experiments of Vanilla

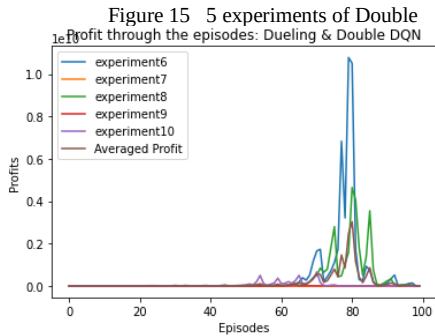
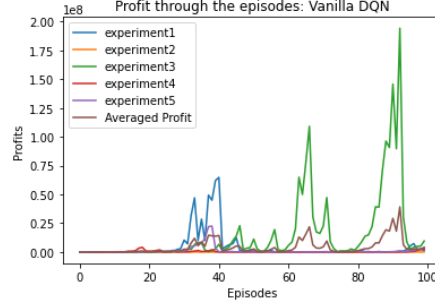


Figure 16 5 experiments of rainbow

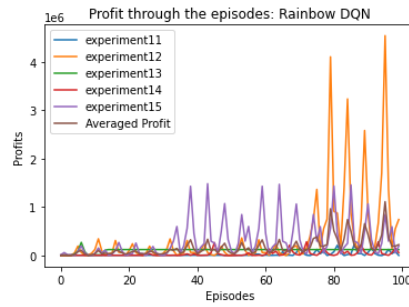
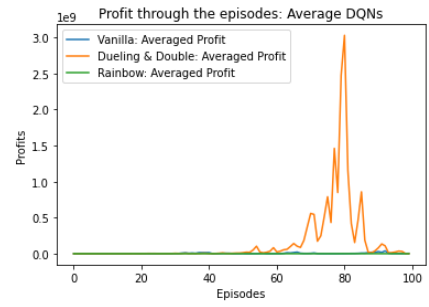


Figure 17 all averaged profit of all DQNs



VIII. ANALYSIS OF THE RESULTS QUANTITATIVELY AND QUALITATIVELY(TASK8)

Overall, any algorithms failed to converge (see figure 14, 15, 16). This is probably because the huger complexity of the stock market trading environment (stock price moves so randomly and prediction is so difficult). Traditionally, many people tried to predict the stock market return by using machine learning algorithms such as LSTM and by using the traditional time series statistic techniques such as VAR model. However, still any model does not predict the stock market return well. Therefore, as the A, B, C graphs show, my DQN type agent also failed to have the stable stock market strategy to earn money in a stable manner, but stock trading market is so challenging environment to learn the optimal strategy, and so the failure of my DQN trader is not surprising.

Even though I failed to have the converging results, I will compare the results with each other. The figure 17 describes the averaged three DQN type agent results (I have 5 experiments in each DQN type agent, and so I took the average for each DQN type agent). It states that dueling and double DQN agent clearly outperforms rather than rainbow and vanilla DQN agents. This result leads to the possibility that n-step, noisy net or prioritized experience memory makes the performance worse, meaning that some of them might not match the stock trading environment. If n-step failed to improve the performance, then I would say that several steps to calculate the reward is not good for my environment. If noisy net failed to improve the performance, then I would say that stochastic exploration strategy is not nice for the stock market environment. Finally, if PER failed to improve the agent performance, then it may suggest that off policy algorithm itself does not match the stock market trading environment, because all DQN type model are off policy algorithms and they use the past experiences and as long as using the past experiences to improve the performance, logically, PER should improve the agent performance well.

On the other hand, I will explain the reason why dueling and double DQN agent has very good performance based on figure 17. First of all, I only run 5 experiments and take the average of them, and so this excellent perform may be by chance. If I consider these good results come from the improvements of DQN, then they are 2 possible stories. One is that dueling improvement very matches the stock market environment. This means that measuring action importance in each state is the good strategy for the stock environment. The other is that double q improvement successfully avoid the overestimation bias in the stock trading market. In my stock trading market, my agent only has 3 actions for each state, and so the agent is possibly more likely to think that this good performance (positive reward) is caused by its nice action. Thus, double q helps my agent to avoid the above mis interpretation (overestimation bias).

In conclusion, for the further research, to see the convergence, I will run off policy algorithm such as dueling and double DQN for the longer episodes and with longer time steps. In addition, I will use on policy algorithms such as PPO, because off policy algorithm itself may not fit the stock trading environment.

IX. APPLY THE RL ALGORITHM OF MY CHOICE TO ONE OF ATARI LEARNING ENVIRONMENT AND BRIEFLY PRESENT THE ALGORITHM AND JUSTIFY MY CHOICE(TASK9)

A. Environment

For this task I selected Spence Invader game as one of Atari environment. The reason of the choice of this game is simply that this is the only game which I have played. In this game, the agent can take 6 actions: Noop, Fire, Right, Left, Right Fire, Left Fire [5]. Thus, the agent action is discrete one, while the observation space is continuous (infinitely many states). When the agent breaks the opponent, the agent gets the instant reward and the game continues until the agent is broken by the opponent. In this game, my DRL agent would like to find the optimal policy to maximize the cumulative reward. The choice of the algorithm is DQN with some improvements such as Dueling DQN. Which improvements are employed is determined though hyper parameter tuning by using tune in ray.

B. Brief presentation of my choice of algorithm from rllib and justification of my choice

I select DQN type algorithm for this task. DQN is the combination of Q-learning and deep neural network. DQN works with discrete action space and continuous state space by approximating Q values through deep neural network. It has several improvements to make the performance better, such as dueling, double q, noisy net, and prioritized experience replay (To see the detailed information of how each improvement works, see section VII B and C).

The reason why I choose DQN type algorithm is that simply I am the most familiar with DQN type algorithms through this course. In addition, my environment has discrete action space, and cumulative reward keeps increasing until the agent is defeated by the opponent. Thus, some advanced on-policy

algorithms are not appropriate for my environment, Space Invader Game. Additionally, in my opinion, using past experiences is huge advantage to learn the future, and so I prefer off- policy algorithms such as DQN to on-policy algorithms.

C. the workflow to justify the best model

To achieve this goal, I execute 2 level hyper parameter (HP) tunings and test the best agent with longer period game to see the convergence, more precisely, on the first stage HP tuning, I will justify which improvements should be added to Vanilla DQN to have the smartest agent in the Space Invader Game. Thus, HPs are as follows: prioritized replay memory, dueling, double, noisy and n-step learning. In sum, 32 HP combinations are tested and the best choice of improvements will be selected based on the best model choice from tune. Due to the computational resource constrains, I execute this tuning for only 20 episodes by using Grid Search.

- Hyper parameters:
- Dueling = [True, False], Double = [True, False], PRM = [True, False], Noisy = [True, False], N-step = [1, 5]

In the next HP tuning stage, the important HPs such as gamma are tuned to polish the DQN agent. For the HPs, gamma, batch size, the number of timesteps in each episode and the frequency of updating target Q value are chosen as HPs. The total HP combinations are 16.

- Hyper parameters:
- Gamma = [0.2 0.8], Batch size = [32, 64], Timesteps = [500, 1000], Frequency of Q update = [250, 500]

Then, after these HP tunings, the best model is selected and the best agent will play for 100 periods to see its long-term performance and reward convergence.

D. Results

Figure 18 Mean reward of All combination in 1st level HP

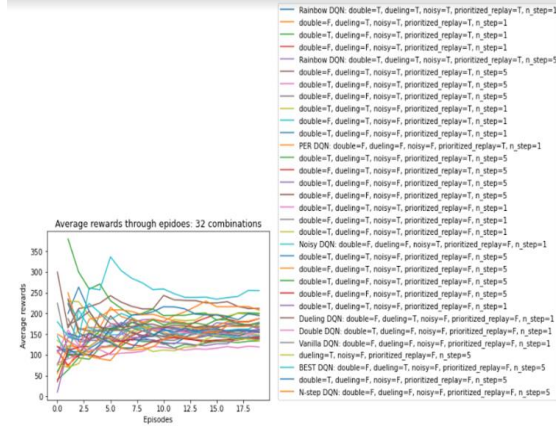


Figure 19 selected 12 results out of 32 results

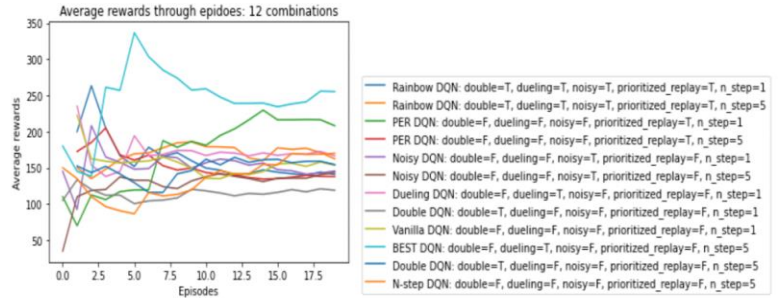


Figure 18 is the total 32 results. The blue line on the top of the graph (see the most above line in the graph) is specified by tune as the best model. Figure 19 is the collection of fundamental results with 1 or 2 improvements. PER DQN is the second best, but for other models, they have almost same results. As the result, the best type of DQN is as follows:

- Double = False, Dueling = True, PRM = False, Noisy = False and N-step = 5

In the next step, with the above best model, other important hyper parameters will be tuned.

Figure 20 Average reward of all HP combinations

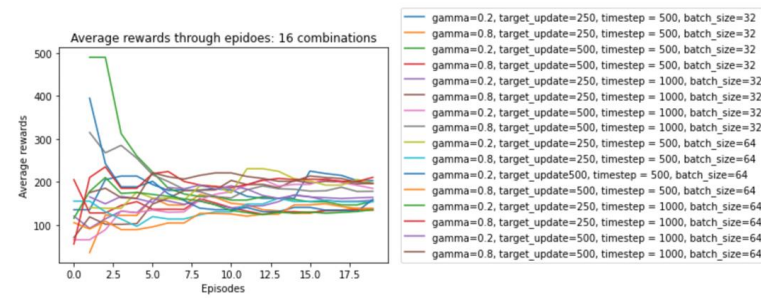


Figure 20 is the collection (16 combinations) of different HP algorithm performances. Overall, any combination has the almost same performance in the long run. Figure 21 is the collection of the results with the same gamma (gamma =0.2). All models seem to have the near performance. Figure 22 is the collection of the results with the same gamma (gamma =0.8). The performance is more different from gamma = 0.2 cases.

Figure 21 the results with the same gamma (gamma =0.2)

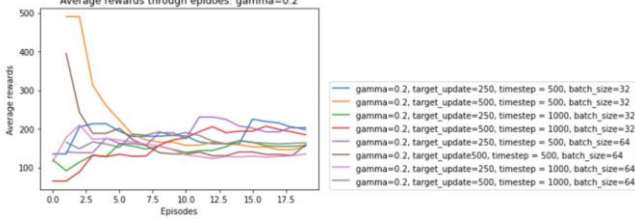


Figure 22 the results with the same gamma (gamma =0.8)

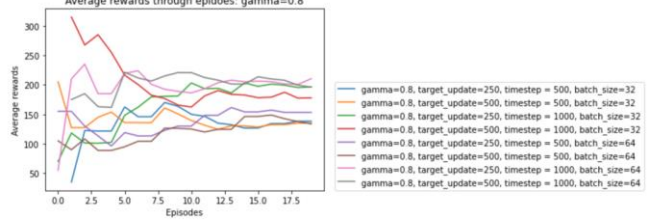


Figure 23 the results with the same update = 250

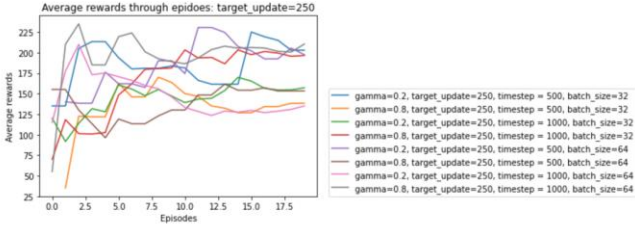


Figure 24 the results with the same update = 500

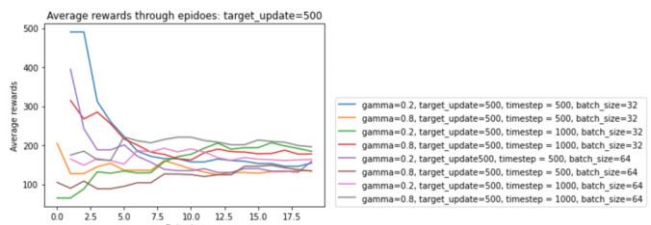


Figure 23 is the collection of the results with the same target update frequency (update = 250). The performances are more different compared with the same gamma cases (see figure 21 and 22). Figure 24 is the collection of the results with the same target update frequency (update = 500) Target update frequency seems to have the impact on the cumulative reward, and small number of updates (update =250) leads to the convergence.

Figure 25 results with the same = 500

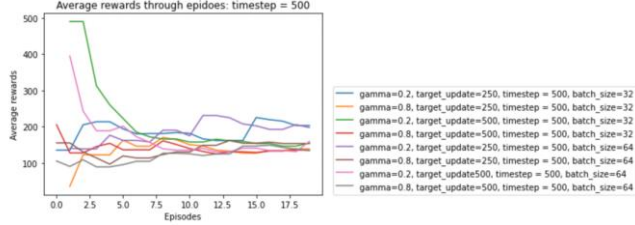


Figure 26 results with the same = 500

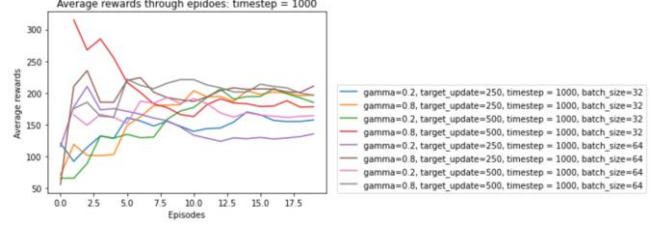


Figure 25 is the collection of the results with the same timesteps (timesteps = 500). All results seem to have near performance. Figure 26 is the collection of the results with the same timesteps (timesteps = 1000). The performances are relatively diverged rather than time steps = 500 cases.

Figure 27 results with the same batch size = 32

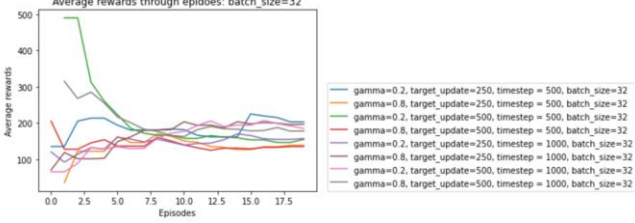


Figure 28 results with the same batch size

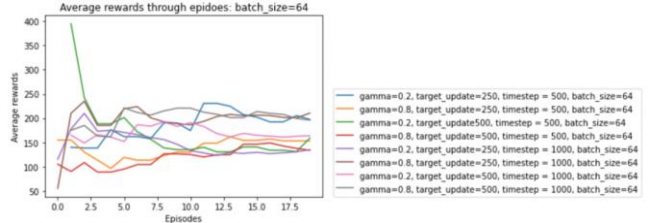


Figure 27 is the collection of the results with the same batch size (batch size = 32). All performances are pretty similar. Figure is the collection of the results with the same batch size (batch size = 64). The performances are more diverged relative to the small batch size cases (batch size =32).

The best tuned model is selected based on the best HP combinations from tune. The best model hp choice is as follows:

- batch size = 64, timesteps = 1000, target q update frequency = 250, gamma = 0.8.

In the last step, I check the long-term performance of the above best model. In sum, the best selected parameters as follows:

- Double = False, Dueling = True, PRM = False, Noisy = False and N-step = 5, batch size = 64, timesteps = 1000, target q update frequency = 250, gamma = 0.8

Figure 29 Mean reward of the best agent through 200 episodes



The graph shows the performance of the best selected model after 2 HP tunings. The best agent plays Space Invader Game for 200 episodes and each episode consist of 1000 time steps. The graph 29 describes that the reward does not perfectly converge, keeping increasing the mean reward gradually.

X. ANALYZE THE RESULTS QUANTITATIVELY AND QUALITATIVELY(TASK10)

In this section, I analyze the results based on the section IX. In the first stage HP tuning, definitely, Dueling and n-step DQN has fat better performance rather than any other models (see figure 18 and 19). In addition, tune responds that Dueling and n-step DQN is the best HP combination model. Thus, by using graphical analysis and tune (in ray) response, Dueling and n-step DQN (5 steps) should be the best perform model. Then, I move onto the second HP tuning for this model.

In the 2nd level HP tuning, overall (see figure 20), any HP combinations perform relatively similar. Thus, to see the difference, I re-describe the results, keeping only one HP but changes other HPs (see figure from 21 to 28).

About gamma, as for gamma = 0.2, all results are very similar (see graph 21), but as for gamma= 0.8, the results are more diverged (see graph 22). However, there is some chance of getting the higher mean reward. Therefore, the tune returns gamma = 0.8 as a part of best HPs. Gamma means the discount factor even in the DQN algorithm. In terms of Space Invader Game, if the agent wants to defeat more opponents to maximize the cumulative reward, then the future return is highly important. This is because gamma =0.8 suggests that the future Q value is more weighted when the Q value is estimated.

About target Q update frequency, as for update= 500, all results are very similar (see figure 24). However, as for update= 250 (see figure 23), the results are more diverged . However, there is some chance of getting the higher mean reward. Therefore, the tune returns update= 250 as a part of best HPs. As we know, the frequency of updating target Q has big impact on the convergence of the cumulative reward. The longer update frequency leads to the faster convergence, while the lower frequency leads to the slower convergence. However, I want the agent to pursue the higher cumulative reward and the tune reply is also the same reply. Thus, my agent may lose the convergence of cumulative reward in the best model (but it is argued later).

About time steps, as for timesteps= 500, all results are very similar (see figure 25), but as for timesteps= 1000, the results are more diverged (see figure 26). However, again, there is some chance of getting the higher mean reward as we observe the case of same gammas and same target q update frequencies. Therefore, the tune returns timesteps= 1000 as a part of best HPs. In Space Invader Game, the game continues until the agent loses; thus, in longer timesteps, the agent can explore the environment and learn the policy more to maximize the cumulative game score. Thus, longer timesteps choice is better.

About batch size, as for batch size = 32, all results are very similar (see figure 27), but as for batch size = 64 (see figure 28), the results look the same. Again, there is some chance of getting the higher mean reward as we observe the cases above. Therefore, the tune returns batch size = 64 as a part of best HPs. As we know, Large batch size is more likely to guarantee that neural network finds the global optimized point, while small batch size may stop at the local optimized point. Therefore, I would say that some models with batch size = 64 found the global optimized point; then, the agent can have better performance.

Finally, about the best model selected by tune, as the graph 29 shows, the mean reward through episodes keeps increasing gradually. However, after the sudden increase of mean reward in around episode 15 to 20, the mean reward keeps over 160. This suggests that the best model agent obtains the good policy in earlier period, and keeps polishing the policy. Therefore, the mean reward keeps increasing gradually. In the longer episode length such as 1000 or 2000, the cumulative reward may converge. However, the Space Invader Game continues the game until the agent loses, which mean that the cumulative reward keeps increasing with longer survive of the agent. Thus, the convergence itself may be a bit difficult rather than another environment. In addition, this result (see graph 29) may come from the high frequent target Q updates (= 250 in my best agent). Therefore, In the future experiment, I am going to use longer frequency of target Q updates and longer episode length for the faster convergence.

References

- [1] Chakole, J. B., Kolhe, M. S., Mahapurush, G. D., Yadav, A., & Kurhekar, M. P. (2021). A Q-learning agent for automated trading in equity stock markets. *Expert Systems with Applications*, 163, 113761.
- [2] King, A. (n.d.). *Stock-trading-environment/data at master · notadamking/stock-trading-environment*. GitHub. Retrieved April 24, 2022, from <https://github.com/notadamking/Stock-Trading-Environment/tree/master/data>
- [3] *Q-LEARNING.PDF*. Sign In. (n.d.). Retrieved April 24, 2022, from <https://moodle.city.ac.uk/course/view.php?id=41658>
- [4] *Lecture 07 Rainbow DQN*. Sign In. (n.d.). Retrieved April 24, 2022, from <https://moodle.city.ac.uk/course/view.php?id=41658>
- [5] Xu, A. Y. (2019, December 27). *Optimized space invaders using deep Q-learning: An implementation in tensorflow 2.0*. Medium. Retrieved April 24, 2022, from <https://towardsdatascience.com/optimized-deep-q-learning-for-automated-atari-space-invaders-an-implementation-in-tensorflow-2-0-80352c744fdc> This is [x] in the text currently.

The contribution of this coursework

I did all the tasks individually, and so I finished 100 % of all tasks from taks 1 to 10.

This script for task 1 to 5 (basic tasks)

In []:

For the implimentation of the algorithm, I basically refere to the lab 04 solution code in our DRL class (called Q-LEARNING LAB CODE SOLUTION)from here:<https://moodle.city.ac.uk/course/view.php?id=41658>

As for the idea of the stock market trading environment, I refere to this stock market traiding implimentation from here:
<https://github.com/notadamking/Stock-Trading-Environment>

Notice: I do not set random seed, and so if you re-run my codes, then you may get a bit different results.

In []:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

In []:

```
#import and show the data
```

In []:

```
df = pd.read_csv('./AAPL.csv')
df = df.sort_values('Date')
```

In []:

```
df.head()
```

In []:

```
for i in range(len(df)-1):
    df.loc[i, "profit"] = df.loc[i+1, 'Close']- df.loc[i, 'Open']
```

In []:

```
df.head()
```

```
In [ ]: # States and Actions
S = ['1', '2', '3', '4']
A = ['buy', 'sell', 'hold']
```

```
In [ ]: # R matrix
R = np.array([[np.nan for a in A] for s in S])
possible_actions = [(0,0), (0,1), (0,2), (1,0), (1, 1), (1,2), (2,0), (2,1), (2,2), (3,0), (3,1), (3,2)]

R[0,0]=0.25#2.5 /10 as holding reward
R[1,0]=0
R[2,0]=-0.28#-2.8/10 as holding cost or penalty
R[3,0]=0
R[0,1]=np.nan
R[1,1]=0
R[2,1]=np.nan
R[3,1]=0
R[0,2]=2.5
R[1,2]=np.nan
R[2,2]=-2.8
R[3,2]=np.nan

# Q matrix
Q = np.zeros(R.shape)#itial conditon
```

```
In [ ]: print(R)#reward matrix
```

```
In [ ]: print(Q)#initial Q matrix
```

This is the main implimentation of Q-learning with my stock trading environment. In this function, you need to specify alpha, gamma, epsilon decay, and obtain the retults to show its performance

```
In [ ]: def RLagent(alpha, gamma, epsilon_decay):
    epsilon = 0.9

    for i in range(len(df)-1):
        df.loc[i, "profit"] = df.loc[i+1, 'Close']- df.loc[i, 'Open']

    # States and Actions
    S = ['0', '1', '2', '3']
    A = ['buy', 'sell', 'hold']

    # R matrix
    R = np.array([[np.nan for a in A] for s in S])
    possible_actions = [(0,0), (0,1), (0,2), (1,0), (1, 1), (1,2), (2,0), (2,1), (2,2), (3,0), (3,1), (3,2)]
```

```

R[0,0]=0.25#2.5 /10 as holding reward
R[1,0]=0
R[2,0]=-0.28#-2.8/10 as holding cost or penalty
R[3,0]=0
R[0,1]=np.nan
R[1,1]=0
R[2,1]=np.nan
R[3,1]=0
R[0,2]=2.5
R[1,2]=np.nan
R[2,2]=-2.8
R[3,2]=np.nan

# Q matrix
Q = np.zeros(R.shape)#itial conditon
initial_badget = 100000

##### for state transition func
State = np.array([[np.nan for s in S] for s in S])
possible_actions = [(0,0), (0,1), (0,2), (1,0), (1, 1), (1,2), (2,0), (2,1), (2,2), (3,0),
(3,1),(3,2)]
#####

State[0,0]=0
State[1,0]=0
State[2,0]=0
State[3,0]=0

State[0,1]=0
State[1,1]=0
State[2,1]=0
State[3,1]=0

State[0,2]=0
State[1,2]=0
State[2,2]=0
State[3,2]=0

State[3,3]=0
State[0,3]=0
State[1,3]=0
State[2,3]=0
State[3,3]=0

#####
q1=[]
q2=[]
q3=[]
q4=[]
q5=[]
q6=[]
q7=[]
q8=[]
q9=[]
q10=[]
q11=[]
q12=[]

```

```
ee=[]
```

```
#####
```

```
for episode in range(2000):
```

```
    initial_date = np.random.randint(4953,5253)# take the initial condition
    state_date=initial_date
```

```
    budget = 1000000000
```

```
    initial_stock_holds = np.random.randint(0,2)
    current_stock_holds=initial_stock_holds
    current_price=df.loc[initial_date, "Close"]
```

```
#Descritize the continous states into 4 states
```

```
    if (df.loc[initial_date, "profit"]>=0) & (current_stock_holds> 0):
        s=0
```

```
    elif (df.loc[initial_date, "profit"]>=0) & (current_stock_holds<= 0):
        s=1
```

```
    elif (df.loc[initial_date, "profit"]<0) & (current_stock_holds > 0):
        s=2
```

```
    elif (df.loc[initial_date, "profit"]<0) & (current_stock_holds <=0):
        s=3
```

```
    print("Starting state is '{}'.format(S[s]))
```

```
#Run the 300 episodes
```

```
    while state_date < 5254:
```

```
        #Action is prohibited in the nan values in R matrix
```

```
        available_actions = np.where(~np.isnan(R[s]))[0]
```

```
        q_values = [Q[s,a] for a in available_actions]
```

```
        best_actions = available_actions[np.where(q_values == np.max(q_values))[0]]
```

```
        best_actions_q_values = [Q[s,x] for x in best_actions]
```

```
        # Epsilon-greedy in each episode
```

```
        if np.random.uniform() > epsilon:
```

```
            a = np.random.choice(available_actions)
```

```
        else:
```

```
            a = np.random.choice(best_actions)
```

```
#Now, a is crrent agent action, and so I can update my budget and based on the agent ac
```

```
#Update current budget to exit if budget <0
```

```
        if a == np.array(0):#anything does not change because the action is "hold"
```

```
            current_stock_holds=current_stock_holds
```

```
            budget = budget
```

```
        elif a ==np.array(1):#buy
```

```
            current_stock_holds +=1
```

```
            budget -= 2.8
```

```
        elif a ==np.array(2):#sell
```

```

        current_stock_holds -=1
        budget += 2.5

# Environment updating
    r = R[s,a]

    s_old = s

    #Need to finish while loop
    state_date +=1

    #Descritize the new state (next day state) into one of 4 types.
    if (df.loc[state_date, "profit"]>=0) & (current_stock_holds> 0):
        s_new=0

    elif (df.loc[state_date, "profit"]>=0) & (current_stock_holds<= 0):
        s_new=1

    elif (df.loc[state_date, "profit"]<0) & (current_stock_holds > 0):
        s_new=2
    elif (df.loc[state_date, "profit"]<0) & (current_stock_holds <=0):
        s_new=3

    #Prohibit the negative number of stock if it happens (in my algorithm, it does not happen)
    if current_stock_holds <= -1:
        break

    #Budget constrain, Negative budget is prohibited.
    if budget <= 0:
        break

# Q value updating
    Q[s_old,a] = Q[s_old,a] + alpha * ( r + gamma * np.max(Q[s_new,:]) - Q[s_old,a])

    #count the state for state transition
    State[s_old, s_new] +=1

    q1=np.append(q1, Q[0,0])
    q2=np.append(q2, Q[1,0])
    q3=np.append(q3, Q[2,0])
    q4=np.append(q4, Q[3,0])
    q5=np.append(q5, Q[0,1])
    q6=np.append(q6, Q[1,1])
    q7=np.append(q7, Q[2,1])
    q8=np.append(q8, Q[3,1])
    q9=np.append(q9, Q[0,2])
    q10=np.append(q10, Q[1,2])
    q11=np.append(q11, Q[2,2])
    q12=np.append(q12, Q[3,2])

#update ε value
    if epsilon >= 0.5:

```

```

        epsilon = epsilon_decay*epsilon

    elif epsilon < 0.5:
        epsilon = (0.1*epsilon_decay)*epsilon

    if epsilon <= 0.01:
        epsilon = epsilon

    ee=np.append(ee,epsilon)

    print('Episode {} finished. Q matrix values:\n{}'.format(episode,Q.round(1)))

    q_vals = [q1,q2,q3,q4,q5,q6,q7,q8,q9,q10,q11,q12]
    q_average= q1+q2+q3+q4+q5+q6+q7+q8+q9+q10+q11+q12
    Q_last = Q

    return q_vals, q_average, Q_last, State

```

The base line experiment corresponds to task 4

```

In [ ]: alpha =0.5
        gamma =0.5
        epsilon_decay=0.9999

        q_vals1, q_average1, Q_last1,State= RLagent(alpha, gamma, epsilon_decay)

```

```

In [ ]: plt.plot(q_average1)

```

```

In [ ]: print(Q_last1.round(1))

```

Point: State transition matrix is manually calculated. Thus, if you rerun the code, then, you need to manually calculate it.

```

In [ ]: print(State)

```

```

In [ ]: print(State/(22+410+24+200+44142+886+34994+659+2433+2876+1945+2256+36747+892+28419+687))

```

```

In [ ]: state_data = [[1.39600995e-04, 2.60165491e-03, 1.52291995e-04, 1.26909995e-03],
                    [2.80103051e-01, 5.62211280e-03, 2.22054419e-01, 4.18168435e-03],
                    [1.54386009e-02, 1.82496573e-02, 1.23419971e-02, 1.43154475e-02],
                    [2.33178080e-01, 5.66018580e-03, 1.80332758e-01, 4.35935834e-03]]

    state_data = pd.DataFrame(state_data)
    state_data.columns = ['s0', 's1', 's2', 's3']
    state_data.index=['s0', 's1', 's2', 's3']
    state_data

```


Lots of experiments with different hyper parameter choices for task 5

2nd experiment with $\alpha=0.2$ and $\gamma=0.5$

```
In [ ]: alpha =0.2
        gamma =0.5
        epsilon_decay=0.9999

        q_vals2, q_average2, Q_last2,State2= RLagent(alpha, gamma, epsilon_decay)

In [ ]: plt.plot(q_average2)

In [ ]: print(Q_last2.round(1))
```

3rd experiment $\alpha = 0.9$ and $\gamma = 0.5$

```
In [ ]: alpha =0.9
        gamma =0.5
        epsilon_decay=0.9999

        q_vals3, q_average3, Q_last3,State3= RLagent(alpha, gamma, epsilon_decay)

In [ ]: plt.plot(q_average3)

In [ ]: print(Q_last3.round(1))
```

4th experiment $\alpha = 0.5$ & $\gamma = 0.2$

```
In [ ]: alpha =0.5
        gamma =0.2
        epsilon_decay=0.9999

        q_vals4, q_average4, Q_last4,State4= RLagent(alpha, gamma, epsilon_decay)

In [ ]: plt.plot(q_average4)

In [ ]: print(Q_last4.round(1))
```

5th experiment alpha = 0.5 & gamma = 0.9

```
In [ ]: alpha =0.5
        gamma =0.9
        epsilon_decay=0.9999

        q_vals5, q_average5, Q_last5,State5= RLagent(alpha, gamma, epsilon_decay)

In [ ]: plt.plot(q_average5)

In [ ]: print(Q_last5.round(1))
```

6th experiment alpha = 0.9 & gamma = 0.9

```
In [ ]: alpha =0.9
        gamma =0.9
        epsilon_decay=0.9999

        q_vals6, q_average6, Q_last6,State6= RLagent(alpha, gamma, epsilon_decay)

In [ ]: plt.plot(q_average6)

In [ ]: print(Q_last6.round(1))
```

7th experiment aplha = 0.2 & gamma = 0.9

```
In [ ]: alpha =0.2
        gamma =0.9
        epsilon_decay=0.9999

        q_vals7, q_average7, Q_last7,State7= RLagent(alpha, gamma, epsilon_decay)

In [ ]: plt.plot(q_average7)

In [ ]: print(Q_last7.round(1))
```

8th experiment aplha = 0.2 & gamma =0.2

```
In [ ]: alpha =0.2
        gamma =0.2
        epsilon_decay=0.9999

        q_vals8, q_average8, Q_last8,State8= RLagent(alpha, gamma, epsilon_decay)
```

```
In [ ]: plt.plot(q_average8)
```

```
In [ ]: print(Q_last8.round(1))
```

9th experiment alpha = 0.9 & gamma = 0.2

```
In [ ]: alpha =0.9
gamma =0.2
epsilon_decay=0.9999

q_vals9, q_average9, Q_last9,State9= RLagent(alpha, gamma, epsilon_decay)
```

```
In [ ]: print(Q_last1)
```

```
In [ ]: print(Q_last5)
```

10th Experiment only for $\alpha = 0.9$, $\gamma = 0.9$ model (best performed model between 1st and 9th experiments) with different epsilon decay

```
In [ ]: alpha =0.9
gamma =0.9
epsilon_decay=0.3

q_vals10, q_average10, Q_last10,State10= RLagent(alpha, gamma, epsilon_decay)
```

```
In [ ]: plt.plot(q_average6, label ="\alpha = 0.9, \gamma=0.9, epsilon_decay = 0.9999")
plt.plot(q_average10, label ="\alpha = 0.9, \gamma=0.9, epsilon_decay = 0.3")
plt.xlabel("Episodes")
plt.ylabel("Averaged Cumulative reward")
plt.title("Different epsilon decay experiment results")

plt.legend()

plt.show()
```

```
In [ ]: #!/refre to how to add legends to my plot from here:https://pythonprogramming.net/legends-tit

plt.plot(q_average1, label ="\alpha = 0.5, \gamma=0.5")
plt.plot(q_average2, label ="\alpha = 0.2, \gamma=0.5")
plt.plot(q_average3, label ="\alpha = 0.9, \gamma=0.5")
plt.plot(q_average4, label ="\alpha = 0.5, \gamma=0.2")
plt.plot(q_average5, label ="\alpha = 0.5, \gamma=0.9")
plt.plot(q_average6, label ="\alpha = 0.9, \gamma=0.9")
plt.plot(q_average7, label ="\alpha = 0.2, \gamma=0.9")
plt.plot(q_average8, label ="\alpha = 0.2, \gamma=0.2")
plt.plot(q_average9, label ="\alpha = 0.9, \gamma=0.2")
plt.plot(q_average10, label ="\alpha = 0.9, \gamma=0.9, epsilon_decay = 0.3")
```

```
plt.xlabel("Episodes")
plt.ylabel("Averaged Cumulative reward")
plt.title("All experiment results")
plt.legend(loc=(1.04,0))
```

```
In [ ]: plt.plot(q_average1, label = " $\alpha = 0.5$ ,  $\gamma = 0.5$ ")
plt.plot(q_average4, label = " $\alpha = 0.5$ ,  $\gamma = 0.2$ ")
plt.plot(q_average5, label = " $\alpha = 0.5$ ,  $\gamma = 0.9$ ")
plt.xlabel("Episodes")
plt.ylabel("Averaged Cumulative reward")
plt.title(" $\alpha = 0.5$  experiment results")

plt.legend()

plt.show()
```

```
In [ ]: plt.plot(q_average2, label = " $\alpha = 0.2$ ,  $\gamma = 0.5$ ")

plt.plot(q_average7, label = " $\alpha = 0.2$ ,  $\gamma = 0.9$ ")
plt.plot(q_average8, label = " $\alpha = 0.2$ ,  $\gamma = 0.2$ ")
plt.xlabel("Episodes")
plt.ylabel("Averaged Cumulative reward")
plt.title(" $\alpha = 0.2$  experiment results")

plt.legend()

plt.show()
```

```
In [ ]: plt.plot(q_average3, label = " $\alpha = 0.9$ ,  $\gamma = 0.5$ ")

plt.plot(q_average6, label = " $\alpha = 0.9$ ,  $\gamma = 0.9$ ")

plt.plot(q_average9, label = " $\alpha = 0.9$ ,  $\gamma = 0.2$ ")
plt.xlabel("Episodes")
plt.ylabel("Averaged Cumulative reward")
plt.title(" $\alpha = 0.9$  experiment results")
plt.legend()

plt.show()
```

```
In [ ]: plt.plot(q_average1, label = " $\alpha = 0.5$ ,  $\gamma = 0.5$ ")
plt.plot(q_average2, label = " $\alpha = 0.2$ ,  $\gamma = 0.5$ ")
plt.plot(q_average3, label = " $\alpha = 0.9$ ,  $\gamma = 0.5$ ")
plt.xlabel("Episodes")
plt.ylabel("Averaged Cumulative reward")
plt.title(" $\gamma = 0.5$  experiment results")

plt.legend()

plt.show()
```

```
In [ ]: plt.plot(q_average4, label = " $\alpha = 0.5$ ,  $\gamma = 0.2$ ")

plt.plot(q_average8, label = " $\alpha = 0.2$ ,  $\gamma = 0.2$ ")
```

```
plt.plot(q_average9, label = " $\alpha = 0.9, \gamma = 0.2$ ")
plt.xlabel("Episodes")
plt.ylabel("Averaged Cumulative reward")
plt.title(" $\gamma = 0.2$  experiment results")
plt.legend()

plt.show()
```

In []:

```
plt.plot(q_average5, label = " $\alpha = 0.5, \gamma = 0.9$ ")
plt.plot(q_average6, label = " $\alpha = 0.9, \gamma = 0.9$ ")
plt.plot(q_average7, label = " $\alpha = 0.2, \gamma = 0.9$ ")
plt.xlabel("Episodes")
plt.ylabel("Averaged Cumulative reward")
plt.title(" $\gamma = 0.9$  experiment results")

plt.legend()

plt.show()
```

In []:

```
plt.plot(q_average1, label = " $\alpha = 0.5, \gamma = 0.5$ ")

plt.xlabel("Episodes")
plt.ylabel("Averaged Cumulative reward")
plt.title("Base experiment result")

plt.legend()

plt.show()
```

In []:

This is the script for task 7 and 8.

First of all, I very much referred to the codes here for the all types of DQN for tasks 7 and 8:<https://github.com/Curt-Park/rainbow-is-all-you-need>

Above link is a github link and it includes the links from Vallira to Rainbow. Just in case, I put the links from Vallira to Rainbow.

(Vanilla)

DQN:<https://colab.research.google.com/github/Curt-Park/rainbow-is-all-you-need/blob/master/01.dqn.ipynb>

Double

DQN:https://colab.research.google.com/github/Curt-Park/rainbow-is-all-you-need/blob/master/02.double_q.ipynb

Proritized Experience Replay

DQN:<https://colab.research.google.com/github/Curt-Park/rainbow-is-all-you-need/blob/master/03.per.ipynb>

Dueling DQN:

<https://colab.research.google.com/github/Curt-Park/rainbow-is-all-you-need/blob/master/04.dueling.ipynb>

Noisy

DQN:<https://colab.research.google.com/github/Curt-Park/rainbow-is-all-you-need/blob/master/05.noisy.ipynb>

[Park/rainbow-is-all-you-need/blob/master/05.noisy_net.ipynb](https://colab.research.google.com/github/CPark/rainbow-is-all-you-need/blob/master/05.noisy_net.ipynb)

n-step

DQN:https://colab.research.google.com/github/CPark/rainbow-is-all-you-need/blob/master/07.n_step_learning.ipynb

Rainbow

DQN:<https://colab.research.google.com/github/CPark/rainbow-is-all-you-need/blob/master/08.rainbow.ipynb>

For me, it is so difficult to make improved DQNs such as PER DQN which works in my environment. Therefore, I borrowed the external Rainbow DQN codes as I noticed above.

The strategy to impliment DQN algorithms is that I borrow Rainbow DQN from the link above, and I simplify the borrowed Rainbow DQN as much as possible, because the borrrwoed Rainbow DQN includes Categorical DQN part, but I do not need it. To simplify the Rainbow DQN, I refered to above links (Proritized Experience Replay DQN, Double DQN, Dueling DQN, Noisy DQN, n-step DQN). Thus, my implimented algorithms are like a patch work from the external codes. In sum, my contribution is to simplify the Rainbow DQN algorithm and adapt it to my environment. I know my score will be reduced because I used external codes alot, but that is what I did to complete task 7 and 8.

The original borrowed rainbow DQN does not work well, because the original one is implemented for Atari environment. For example, the original one does not read my environment step function. In addition, the original one does not have the episode loop in the function train(). Thus, I makes this loop to show my environment results in order to describe the performance of the my modified algorithms. This is another contribution. In addition, the original one cannot store the result data, and so I implemented the codes to extract the data (results) to make the graphs. This is another contribution.

For Vanilla and Dueling & Double DQN implementations, I modified my arranged Rainbow DQNs (specifically, the unnecessary part such as PER part and n-step part are omitted) to make them. In addition, to simplify the modified Vanilla and Dueling & Double DQNs, I somewhat referred to Vanilla, Dueling and Double DQN implementations from above links.

As for my environment, I also very referred to the codes here:

<https://github.com/notadamking/Stock-Trading-Environment>

However, I modified the borrowed environment codes some what in order for my environment to work with my implemented DQNs.

As for the data, I took it from the above link (<https://github.com/notadamking/Stock-Trading-Environment>)

The explanation of the original environment codes are here: <https://towardsdatascience.com/creating-a-custom-openai-gym-environment-for-stock-trading-be532be3910e>

Notice: I do not set up random seed, and so if you re-run my code, then you may get the different values of results.

Preprocessing

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

```
In [ ]: import os

# TODO: Fill in the Google Drive path where you uploaded the lab materials
# Example: GOOGLE_DRIVE_PATH_AFTER_MYDRIVE = 'Colab Notebooks/Lab materials 01-2021'

GOOGLE_DRIVE_PATH_AFTER_MYDRIVE = 'Colab Notebooks/DRL'
GOOGLE_DRIVE_PATH = os.path.join('drive', 'My Drive', GOOGLE_DRIVE_PATH_AFTER_MYDRIVE)
print(os.listdir(GOOGLE_DRIVE_PATH))
```

```
In [ ]: %load_ext autoreload
%autoreload 2
```

```
In [ ]: import sys
CODE_PATH = os.path.join(GOOGLE_DRIVE_PATH, 'my_env4')
sys.path.append(CODE_PATH)
```

```
In [ ]: import sys
IN_COLAB = "google.colab" in sys.modules

#they are probably nessarily to run my code, but I reserve these codes to run my DQNs safely.
if IN_COLAB:
    !apt install python-opengl
    !apt install ffmpeg
    !apt install xvfb
    !pip install pyvirtualdisplay
```

```
!pip install -U gym
from pyvirtualdisplay import Display

# Start virtual display
dis = Display(visible=0, size=(400, 400))
dis.start()
```

```
In [ ]:
import json
import datetime as dt
import pandas as pd
import matplotlib.pyplot as plt
```

<https://www.geeksforgeeks.org/ways-to-import-csv-files-in-google-colab/>

Here and the below codes to import data from what I have in local pc, I followed the way to import the csv.file from above link!!

```
In [ ]:
#https://www.geeksforgeeks.org/ways-to-import-csv-files-in-google-colab/
#Here and the below codes to import data from what I have in local pc, I followed the way to impo

#Notice: I uploaded my data from my local laptop to google colab to use my data.
#Thus, to execute this part of codes, you need to have the data in your local PC and then select t
#Therefore, I will upload the data file with the codes (probably...)
from google.colab import files
uploaded = files.upload()
```

```
In [ ]:
import pandas as pd
import io

df = pd.read_csv(io.BytesIO(uploaded['AAPL.csv']))
df = df.sort_values('Date')

#If you run the code in jupyter notebook, then you use
# df = pd.read_csv("AAPL.csv") with locating AAPL.csv file in the the same directly of this script
```

```
In [ ]:
import math
import os
import random
from collections import deque
from typing import Deque, Dict, List, Tuple

import gym
import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from IPython.display import clear_output
```

```

from torch.nn.utils import clip_grad_norm_

# download segment tree module : this probably nesseralily, but to run my code safly, I leave the
if IN_COLAB:
    !wget https://raw.githubusercontent.com/curt-park/rainbow-is-all-you-need/master/segment_

from segment_tree import MinSegmentTree, SumSegmentTree

```

Import Environment

```

In [ ]:
from Custom_env_4 import StockTradingEnv
env = StockTradingEnv(df)

```

Vanillira DQN

```

In [ ]:
# Replya Buffer

```

```

In [ ]:
class ReplayBuffer:

    def __init__(self, obs_dim: int, size: int, batch_size: int = 32):
        self.obs_buf = np.zeros([size, obs_dim], dtype=np.float32)
        self.next_obs_buf = np.zeros([size, obs_dim], dtype=np.float32)
        self.acts_buf = np.zeros([size], dtype=np.float32)
        self.rews_buf = np.zeros([size], dtype=np.float32)
        self.done_buf = np.zeros(size, dtype=np.float32)
        self.max_size, self.batch_size = size, batch_size
        self.ptr, self.size, = 0, 0

    def store(
        self,
        obs: np.ndarray,
        act: np.ndarray,
        rew: float,
        next_obs: np.ndarray,
        done: bool,
    ):
        self.obs_buf[self.ptr] = obs
        self.next_obs_buf[self.ptr] = next_obs
        self.acts_buf[self.ptr] = act
        self.rews_buf[self.ptr] = rew
        self.done_buf[self.ptr] = done
        self.ptr = (self.ptr + 1) % self.max_size
        self.size = min(self.size + 1, self.max_size)

    def sample_batch(self) -> Dict[str, np.ndarray]:
        idxs = np.random.choice(self.size, size=self.batch_size, replace=False)
        return dict(obs=self.obs_buf[idxs],
                    next_obs=self.next_obs_buf[idxs],
                    acts=self.acts_buf[idxs],
                    rews=self.rews_buf[idxs],
                    done=self.done_buf[idxs])

    def __len__(self) -> int:
        return self.size

```

```
In [ ]: #Network
```

```
In [ ]: class Network_Vanillira(nn.Module):
    def __init__(self, in_dim: int, out_dim: int):

        super(Network_Vanillira, self).__init__()

        self.layers = nn.Sequential(
            nn.Linear(in_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, out_dim)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """Forward method implementation."""
        return self.layers(x)
```

```
In [ ]: # Agent
```

```
In [ ]: class DQNAgent:
    """Vanilla DQN

    env (gym.Env): To read my stock environment
    memory: replay memory to store transitions
    batch_size (int): batch size for sampling
    target_update (int): the update frequency of target q
    gamma: discount rate
    dqn (Network): model to train and select actions
    dqn_target (Network): target model to update
    optimizer (torch.optim): optimizer for training dqn
    transition (list): transition information including state, action, reward, next_state
    support (torch.Tensor): support for categorical dqn
    """

    def __init__(
        self,
        env: gym.Env,
        memory_size: int,
        batch_size: int,
        target_update: int,
        epsilon_decay: float = 1.0,
        max_epsilon: float = 1.0,
        min_epsilon: float = 0.1,
        gamma: float = 0.99,
    ):
        """ initialization
        env (gym.Env): To read my stock environment
        memory: replay memory to store transitions
        batch_size (int): batch size for sampling
        target_update (int): the update frequency of target q
        epsilon_decay (float): step size to decrease epsilon
        lr (float): learning rate
        max_epsilon (float): max value of epsilon
        min_epsilon (float): min value of epsilon
        """
```

```

        gamma (float): discount factor
    """
    obs_dim = env.observation_space.shape[0]
    action_dim = env.action_space.n

    self.env = env
    self.memory = ReplayBuffer(obs_dim, memory_size, batch_size)
    self.batch_size = batch_size
    self.epsilon = max_epsilon
    self.epsilon_decay = epsilon_decay
    self.max_epsilon = max_epsilon
    self.min_epsilon = min_epsilon
    self.target_update = target_update
    self.gamma = gamma

    # device specification
    self.device = torch.device(
        "cuda" if torch.cuda.is_available() else "cpu"
    )
    print(self.device)

    # networks: dqn and dqn_target
    self.dqn = Network_Vanillira(obs_dim, action_dim).to(self.device)
    self.dqn_target = Network_Vanillira(obs_dim, action_dim).to(self.device)
    self.dqn_target.load_state_dict(self.dqn.state_dict())
    self.dqn_target.eval()

    # optimizer
    self.optimizer = optim.Adam(self.dqn.parameters())

    # transition to store in memory
    self.transition = list()

# epsilon greedy policy!!
    def policy(self, state: np.ndarray) -> np.ndarray:
        """Select an action from the input state."""

        if self.epsilon > np.random.random():
            selected_action = self.env.action_space.sample()
        else:
            selected_action = self.dqn(
                torch.FloatTensor(state).to(self.device)
            ).argmax()
            selected_action = selected_action.detach().cpu().numpy()

        if not self.is_test:
            self.transition = [state, selected_action]

        self.epsilon = self.epsilon * self.epsilon_decay #0.9999 is the constant decay paramete

        if self.epsilon < 0.02: #this is the minimum threshold of epsilon value
            self.epsilon = self.epsilon

        return selected_action

    def update_model(self) -> torch.Tensor:

```

```

# PER needs beta to calculate weights (this will be used for Rainbow)
samples = self.memory.sample_batch(self.beta)
weights = torch.FloatTensor(
    samples["weights"].reshape(-1, 1)
).to(self.device)
indices = samples["indices"]

# 1-step Learning loss (because normal DQN)
elementwise_loss = self._compute_dqn_loss(samples, self.gamma)

self.optimizer.zero_grad()
loss.backward()
clip_grad_norm_(self.dqn.parameters(), 10.0)
self.optimizer.step()

return loss.item()

#num_frames = # of episode, at this moment, I set 100 episode
#Here, I train the agent
def train(self, num_frames: int, num_episode: int = int):

    self.is_test = False
    memory_episode=[]
    losses = []
    loss = np.nan

    state = self.env.reset()
    for i in range(num_episode):
        #this is added by myself
        #here, this is the same as obs = env.reser()!!
        update_cnt = 0
        #this is collected to show the graph!!
        scores = [] #this is collected to show the graph
        score = 0 #Initial reward

        #this is for loop = # of steps in each episode
        #this should be length of the data!!
        for frame_idx in range(1, num_frames + 1):
            action = self.policy(state)
            next_state, reward, done, _ = self.env.step(action)

            state = next_state

        # if episode ends
        if done:
            state = self.env.reset()
            scores.append(score)
            score = 0
            #added

```

```

        #profit = 0

    # if training is ready
    if len(self.memory) >= self.batch_size:
        loss = self.update_model()
        update_cnt += 1

    # if the target q update is needed
    if update_cnt % self.target_update == 0:
        self._target_hard_update()

    #this makes the average_values in each episode. A
    episode_output = self.env.render()
    memory_episode.append(episode_output)
    losses.append(loss)
    profit, max_net_worth, current_step, current_balance, shares_held, total_shares_sold,

    df_episode_output=pd.DataFrame(memory_episode)#
    df_episode_output.to_pickle("./dummy.pkl")
    df_episode_loss=pd.DataFrame(losses)
    df_episode_loss.to_pickle("./dummy2.pkl")

    self.env.close()

    return df_episode_output

def _compute_dqn_loss(self, samples: Dict[str, np.ndarray], gamma: float) -> torch.Tensor
    """Return the loss values"""
    device = self.device # device specification
    state = torch.FloatTensor(samples["obs"]).to(device)
    next_state = torch.FloatTensor(samples["next_obs"]).to(device)
    action = torch.LongTensor(samples["acts"]).to(device)
    reward = torch.FloatTensor(samples["rews"].reshape(-1, 1)).to(device)
    done = torch.FloatTensor(samples["done"].reshape(-1, 1)).to(device)

    curr_q_value = self.dqn(state).gather(1, action)
    next_q_value = self.dqn_target(next_state).gather( # Double DQN
        1, self.dqn(next_state).argmax(dim=1, keepdim=True)
    ).detach()
    mask = 1 - done
    target = (reward + self.gamma * next_q_value * mask).to(self.device)

    # calculate dqn loss here
    loss = F.smooth_l1_loss(curr_q_value, target)

    return loss
# Target update
def _target_hard_update(self):
    self.dqn_target.load_state_dict(self.dqn.state_dict())

```

Experiment 1 of Vanillra

In []:

```
# parameters
num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent(env, memory_size, batch_size, target_update)
```

In []:

```
agent.train(num_frames,num_episode)
```

In []:

```
result1 = pd.read_pickle("./dummy.pkl")
print(result1)

#I could not put the column names for result1 in google colab. thus, I explain them below.
#[profit, max_net_worth,current_step,current_balance,shares_held,total_shares_sold,cost_base]
#profit = column 0 and the order above is correspond to the column names from 0 to 8.
#Thus, new_worth = column 8

#Anyway, I only use column 0 information (profit informatioin), and so this is enough explanation.
```

experiment 2

In []:

```
# parameters
num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent(env, memory_size, batch_size, target_update)
```

In []:

```
agent.train(num_frames,num_episode)
```

In []:

```
result2 = pd.read_pickle("./dummy.pkl")
print(result2)

#I could not put the column names for result in google colab. thus, I explain them below.
#[profit, max_net_worth,current_step,current_balance,shares_held,total_shares_sold,cost_base]
#profit = column 0 and the order above is correspond to the column names from 0 to 8.
#Thus, new_worth = column 8

#Anyway, I only use column 0 information (profit informatioin), and so this is enough explanation.
```

experiment 3

In []:

```
# parameters
```



```

num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent(env, memory_size, batch_size, target_update)
agent.train(num_frames,num_episode)

```

In []:

```

result3 = pd.read_pickle("./dummy.pkl")
print(result3)

#I could not put the column names for result in google colab. thus, I explain them below.
#[profit, max_net_worth,current_step,current_balance,shares_held,total_shares_sold,cost_base]
#profit = column 0 and the order above is correspond to the column names from 0 to 8.
#Thus, new_worth = column 8

#Anyway, I only use column 0 information (profit informatioin), and so this is enough explanation.

```

experiment 4

In []:

```

# parameters
num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent(env, memory_size, batch_size, target_update)
agent.train(num_frames,num_episode)

```

In []:

```

result4 = pd.read_pickle("./dummy.pkl")
print(result4)

```

experiment 5

In []:

```

# parameters
num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent(env, memory_size, batch_size, target_update)
agent.train(num_frames,num_episode)
result5 = pd.read_pickle("./dummy.pkl")
print(result5)

```

In []:

Dueling and Double DQN

In []:

For replay memory, I use the same code of Vanillira

In []:

```
class Network_Dueling(nn.Module):
    def __init__(
        self,
        in_dim: int,
        out_dim: int,
    ):
        """Initialization."""
        super(Network_Dueling, self).__init__()
        self.feature_layer = nn.Sequential(
            nn.Linear(in_dim, 128),
            nn.ReLU(),
        )

        # set advantage layer for dueling
        self.advantage_layer = nn.Sequential(
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, out_dim),
        )

        # set value layer for dueling
        self.value_layer = nn.Sequential(
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, 1),
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """Do forward for q"""
        feature = self.feature_layer(x)

        value = self.value_layer(feature)
        advantage = self.advantage_layer(feature)

        q = value + advantage - advantage.mean(dim=-1, keepdim=True)

        return q
```

In []:

agent

In []:

```
class DQNAgent2:
    """DQN Agent interacting with environment.

    Attribute:
        env (gym.Env): To read my stock environment
        memory: replay memory to store transitions
        batch_size (int): batch size for sampling
```

```

target_update (int): the update frequency of target q
gamma: discount rate
dqn (Network): model to train and select actions
dqn_target (Network): target model to update
optimizer (torch.optim): optimizer for training dqn
transition (list): transition information including state, action, reward, next_state
memory_n (ReplayBuffer): n-step replay buffer
"""

def __init__(
    self,
    env: gym.Env,
    memory_size: int,
    batch_size: int,
    target_update: int,
    epsilon_decay: float = 1.0,
    max_epsilon: float = 1.0,
    min_epsilon: float = 0.1,
    gamma: float = 0.99
):
    """Initialization.

    Args:
        env (gym.Env): To read my stock environment
        memory: replay memory to store transitions
        batch_size (int): batch size for sampling
        target_update (int): the update frequency of target q
        gamma: discount rate
        prior_eps (float): guarantees every transition can be sampled
    """
    obs_dim = env.observation_space.shape[0]
    action_dim = env.action_space.n

    self.env = env
    self.memory = ReplayBuffer(obs_dim, memory_size, batch_size)
    self.batch_size = batch_size
    self.epsilon = max_epsilon
    self.epsilon_decay = epsilon_decay
    self.max_epsilon = max_epsilon
    self.min_epsilon = min_epsilon
    self.target_update = target_update
    self.gamma = gamma

    # device: device specification
    self.device = torch.device(
        "cuda" if torch.cuda.is_available() else "cpu"
    )
    print(self.device)

    # epsilon greedy policy!!
    def policy(self, state: np.ndarray) -> np.ndarray:
        """Select an action from the input state."""

        if self.epsilon > np.random.random():
            selected_action = self.env.action_space.sample()
        else:
            selected_action = self.dqn(
                torch.FloatTensor(state).to(self.device)
            ).argmax()
            selected_action = selected_action.detach().cpu().numpy()

```

```

if not self.is_test:
    self.transition = [state, selected_action]

self.epsilon = self.epsilon*self.epsilon_decay#0.9999 is the constant decay parameter

if self.epsilon < 0.02:#this is the minimum threshold of epsilon value
    self.epsilon = self.epsilon

return selected_action

# networks: dqn, dqn_target from network
self.dqn = Network_Dueling(obs_dim, action_dim).to(self.device)
self.dqn_target = Network_Dueling(obs_dim, action_dim).to(self.device)
self.dqn_target.load_state_dict(self.dqn.state_dict())
self.dqn_target.eval()

# optimizer for neural network
self.optimizer = optim.Adam(self.dqn.parameters())

# transition to store in memory
self.transition = list()

```

```

def update_model(self) -> torch.Tensor:
    """Loss calculation by using gradient descent."""
    # PER needs beta to calculate weights
    samples = self.memory.sample_batch(self.beta)
    weights = torch.FloatTensor(
        samples["weights"].reshape(-1, 1)
    ).to(self.device)
    indices = samples["indices"]

    # 1-step Learning loss
    elementwise_loss = self._compute_dqn_loss(samples, self.gamma)

    self.optimizer.zero_grad()
    loss.backward()
    clip_grad_norm_(self.dqn.parameters(), 10.0)
    self.optimizer.step()

    return loss.item()

```

#num_frames = # of episode, at this moment, I set 200 episode

```

def train(self, num_frames: int, num_episode: int ):#plotting_interval: int = 1 should be
    """Train the agent."""
    self.is_test = False
    memory_episode=[]
    losses = []
    loss = np.nan

```

```

state = self.env.reset()
for i in range(num_episode):

    update_cnt = 0
    #this is collected to show the graph!!
    scores = [] #this is collected to show the graph
    score = 0 #Initial reward

    #this is for loop = # of steps in each episode
    #this should be length of the data!!
    for frame_idx in range(1, num_frames + 1):
        action = self.policy(state)
        next_state, reward, done, _ = self.env.step(action)

        state = next_state

    # if episode ends (my environment is rare)
    if done:
        state = self.env.reset()
        scores.append(score)
        score = 0

    # if training is ready
    if len(self.memory) >= self.batch_size:
        loss = self.update_model()
        update_cnt += 1

    # if the target Q is needed
    if update_cnt % self.target_update == 0:
        self._target_hard_update()

    #this makes the average_values in each episode.
    episode_output = self.env.render()
    memory_episode.append(episode_output) #added
    losses.append(loss)
    profit, max_net_worth, current_step, current_balance, shares_held, total_shares_sold,

    df_episode_output=pd.DataFrame(memory_episode) #
    df_episode_output.to_pickle("./dummy.pkl")
    df_episode_loss=pd.DataFrame(losses)
    df_episode_loss.to_pickle("./dummy2.pkl")

    #df_episode_output.to_csv()

    self.env.close()

```

```

#profit= memory_episode[0]
return df_episode_output # added

def _compute_dqn_loss(self, samples: Dict[str, np.ndarray], gamma: float) -> torch.Tensor
    """Return DQN loss values"""
    device = self.device # for shortening the following lines
    state = torch.FloatTensor(samples["obs"]).to(device)
    next_state = torch.FloatTensor(samples["next_obs"]).to(device)
    action = torch.LongTensor(samples["acts"]).to(device)
    reward = torch.FloatTensor(samples["rews"].reshape(-1, 1)).to(device)
    done = torch.FloatTensor(samples["done"].reshape(-1, 1)).to(device)

    curr_q_value = self.dqn(state).gather(1, action)
    next_q_value = self.dqn_target(next_state).gather( # Double DQN
        1, self.dqn(next_state).argmax(dim=1, keepdim=True)
    ).detach()
    mask = 1 - done
    target = (reward + self.gamma * next_q_value * mask).to(self.device)

    # calculate dqn loss
    loss = F.smooth_l1_loss(curr_q_value, target)

    return loss

#Q target update is separately executed from q value (estimated value) update.
#Here, this is dueling and DQN
def _target_hard_update(self):
    """target update """
    self.dqn_target.load_state_dict(self.dqn.state_dict())

```

experiment6

In []:

```

# parameters
num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent2(env, memory_size, batch_size, target_update)
agent.train(num_frames,num_episode)
result6 = pd.read_pickle("./dummy.pkl")
print(result6)
#

```

experiment7

In []:

```

# parameters

```

```
num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent2(env, memory_size, batch_size, target_update)
agent.train(num_frames,num_episode)
result7 = pd.read_pickle("./dummy.pkl")
print(result7)
```

In []:

experiment8

In []:

```
# parameters
num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent2(env, memory_size, batch_size, target_update)
agent.train(num_frames,num_episode)
result8 = pd.read_pickle("./dummy.pkl")
print(result8)
```

experiment 9

In []:

```
# parameters
num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent(env, memory_size, batch_size, target_update)
agent.train(num_frames,num_episode)
result9 = pd.read_pickle("./dummy.pkl")
print(result9)
```

experiment 10

In []:

```
# parameters
num_frames = 1000
num_episode= 100
memory_size = 500
```

```

batch_size = 64
target_update = 200

# train
agent = DQNAgent2(env, memory_size, batch_size, target_update)
agent.train(num_frames, num_episode)
result10 = pd.read_pickle("./dummy.pkl")
print(result10)

```

Rainbow DQN

In []: *# use another replay buffer, but only difference is that this is for pripritized replay buffer working*

```

In [ ]: class ReplayBuffer:
        """this is used for Pripritized replay buffer!!"""

        def __init__(
            self,
            obs_dim: int,
            size: int,
            batch_size: int = 32,
            n_step: int = 1,
            gamma: float = 0.99
        ):
            self.obs_buf = np.zeros([size, obs_dim], dtype=np.float32)
            self.next_obs_buf = np.zeros([size, obs_dim], dtype=np.float32)
            self.acts_buf = np.zeros([size], dtype=np.float32)
            self.rews_buf = np.zeros([size], dtype=np.float32)
            self.done_buf = np.zeros(size, dtype=np.float32)
            self.max_size, self.batch_size = size, batch_size
            self.ptr, self.size, = 0, 0

            # for N-step Learning(because this is Rainbow DQN!)
            self.n_step_buffer = deque(maxlen=n_step)
            self.n_step = n_step
            self.gamma = gamma

        def store(
            self,
            obs: np.ndarray,
            act: np.ndarray,
            rew: float,
            next_obs: np.ndarray,
            done: bool,
        ) -> Tuple[np.ndarray, np.ndarray, float, np.ndarray, bool]:
            transition = (obs, act, rew, next_obs, done)
            self.n_step_buffer.append(transition)

            # if single step transition is not ready
            if len(self.n_step_buffer) < self.n_step:
                return ()

            # create n-step transition, in my case 3 steps!
            rew, next_obs, done = self._get_n_step_info(
                self.n_step_buffer, self.gamma
            )

```



```

obs, act = self.n_step_buffer[0][:2]

self.obs_buf[self.ptr] = obs
self.next_obs_buf[self.ptr] = next_obs
self.acts_buf[self.ptr] = act
self.rews_buf[self.ptr] = rew
self.done_buf[self.ptr] = done
self.ptr = (self.ptr + 1) % self.max_size
self.size = min(self.size + 1, self.max_size)

return self.n_step_buffer[0]

def sample_batch(self) -> Dict[str, np.ndarray]:
    idxs = np.random.choice(self.size, size=self.batch_size, replace=False)

    return dict(
        obs=self.obs_buf[idxs],
        next_obs=self.next_obs_buf[idxs],
        acts=self.acts_buf[idxs],
        rews=self.rews_buf[idxs],
        done=self.done_buf[idxs],
        # for N-step Learning
        indices=idxs,
    )

def sample_batch_from_idx(
    self, idxs: np.ndarray
) -> Dict[str, np.ndarray]:
    # for N-step Learning, in my case 3 steps
    return dict(
        obs=self.obs_buf[idxs],
        next_obs=self.next_obs_buf[idxs],
        acts=self.acts_buf[idxs],
        rews=self.rews_buf[idxs],
        done=self.done_buf[idxs],
    )

def _get_n_step_info(
    self, n_step_buffer: Deque, gamma: float
) -> Tuple[np.int64, np.ndarray, bool]:
    """Return n step (3 steps) reward, next_obs, and done"""
    # the last transition information
    rew, next_obs, done = n_step_buffer[-1][-3:]

    for transition in reversed(list(n_step_buffer[:-1])):
        r, n_o, d = transition[-3:]

        rew = r + gamma * rew * (1 - d)
        next_obs, done = (n_o, d) if d else (next_obs, done)

    return rew, next_obs, done

def __len__(self) -> int:
    return self.size

```

In []:

```

class PrioritizedReplayBuffer(ReplayBuffer):
    """Prioritized Replay buffer. This works with replay buffer, and so you need to read repl

    Attributes:
        max_priority (float): max priority to get distinct states

```

```

tree_ptr (int): next index of tree
alpha (float): alpha parameter for prioritized replay buffer
sum_tree (SumSegmentTree): sum tree for prior
min_tree (MinSegmentTree): min tree for min prior to get max weight

"""

def __init__(
    self,
    obs_dim: int,
    size: int,
    batch_size: int = 32,
    alpha: float = 0.6,
    n_step: int = 1,
    gamma: float = 0.99,
):
    """Here, do initialization"""
    assert alpha >= 0

    super(PrioritizedReplayBuffer, self).__init__(
        obs_dim, size, batch_size, n_step, gamma
    )
    self.max_priority, self.tree_ptr = 1.0, 0
    self.alpha = alpha

    # capacity should be positive, and tree_capacity is needed by multiplied by 2
    tree_capacity = 1
    while tree_capacity < self.max_size:
        tree_capacity *= 2

    self.sum_tree = SumSegmentTree(tree_capacity)
    self.min_tree = MinSegmentTree(tree_capacity)

def store(
    self,
    obs: np.ndarray,
    act: int,
    rew: float,
    next_obs: np.ndarray,
    done: bool,
) -> Tuple[np.ndarray, np.ndarray, float, np.ndarray, bool]:
    """Here, I store the experiemnce and prioritizes!!"""
    transition = super().store(obs, act, rew, next_obs, done)

    if transition:
        self.sum_tree[self.tree_ptr] = self.max_priority ** self.alpha
        self.min_tree[self.tree_ptr] = self.max_priority ** self.alpha
        self.tree_ptr = (self.tree_ptr + 1) % self.max_size

    return transition

def sample_batch(self, beta: float = 0.4) -> Dict[str, np.ndarray]:
    """Do sample experiences of each butch"""
    assert len(self) >= self.batch_size
    assert beta > 0

    indices = self._sample_proportional()

    obs = self.obs_buf[indices]
    next_obs = self.next_obs_buf[indices]
    acts = self.acts_buf[indices]
    rews = self.rews_buf[indices]

```

```

done = self.done_buf[indices]
weights = np.array([self._calculate_weight(i, beta) for i in indices])

return dict(
    obs=obs,
    next_obs=next_obs,
    acts=acts,
    rews=rews,
    done=done,
    weights=weights,
    indices=indices,
)

def update_priorities(self, indices: List[int], priorities: np.ndarray):
    """Here, I update the priorities of transitions (of carese sampled transitions)"""
    assert len(indices) == len(priorities)

    for idx, priority in zip(indices, priorities):
        assert priority > 0
        assert 0 <= idx < len(self)

        self.sum_tree[idx] = priority ** self.alpha
        self.min_tree[idx] = priority ** self.alpha

        self.max_priority = max(self.max_priority, priority)

def _sample_proportional(self) -> List[int]:
    """I sample indices based on proportions."""
    indices = []
    p_total = self.sum_tree.sum(0, len(self) - 1)
    segment = p_total / self.batch_size

    for i in range(self.batch_size):
        a = segment * i
        b = segment * (i + 1)
        upperbound = random.uniform(a, b)
        idx = self.sum_tree.retrieve(upperbound)
        indices.append(idx)

    return indices

def _calculate_weight(self, idx: int, beta: float):
    """Calculate the weight of the experience at idx. This makes how pripritized the experience
    # obtain max weight
    p_min = self.min_tree.min() / self.sum_tree.sum()
    max_weight = (p_min * len(self)) ** (-beta)

    # calculate weights for the priprity level!
    p_sample = self.sum_tree[idx] / self.sum_tree.sum()
    weight = (p_sample * len(self)) ** (-beta)
    weight = weight / max_weight

    return weight

```

```
In [ ]: #noisy layer
```

```
In [ ]: class NoisyLinear(nn.Module):
        """This is Noisy linear to make the noisy nural network.
```

```

Attributes:
    in_features (int): input size of linear module
    out_features (int): output size of linear module
    std_init (float): initial std value
    weight_mu (nn.Parameter): mean value weight parameter
    weight_sigma (nn.Parameter): std value weight parameter
    bias_mu (nn.Parameter): mean value bias parameter
    bias_sigma (nn.Parameter): std value bias parameter

"""

def __init__(
    self,
    in_features: int,
    out_features: int,
    std_init: float = 0.5,
):
    """Here, do initialization"""
    super(NoisyLinear, self).__init__()

    self.in_features = in_features
    self.out_features = out_features
    self.std_init = std_init

    self.weight_mu = nn.Parameter(torch.Tensor(out_features, in_features))
    self.weight_sigma = nn.Parameter(
        torch.Tensor(out_features, in_features)
    )
    self.register_buffer(
        "weight_epsilon", torch.Tensor(out_features, in_features)
    )

    self.bias_mu = nn.Parameter(torch.Tensor(out_features))
    self.bias_sigma = nn.Parameter(torch.Tensor(out_features))
    self.register_buffer("bias_epsilon", torch.Tensor(out_features))

    self.reset_parameters()
    self.reset_noise()

def reset_parameters(self):
    """Reset trainable network parameters. use factorized gaussian noise."""
    mu_range = 1 / math.sqrt(self.in_features)
    self.weight_mu.data.uniform_(-mu_range, mu_range)
    self.weight_sigma.data.fill_(
        self.std_init / math.sqrt(self.in_features)
    )
    self.bias_mu.data.uniform_(-mu_range, mu_range)
    self.bias_sigma.data.fill_(
        self.std_init / math.sqrt(self.out_features)
    )

def reset_noise(self):
    """construct new noise here."""
    epsilon_in = self.scale_noise(self.in_features)
    epsilon_out = self.scale_noise(self.out_features)

    # outer product
    self.weight_epsilon.copy_(epsilon_out.ger(epsilon_in))
    self.bias_epsilon.copy_(epsilon_out)

```

```

def forward(self, x: torch.Tensor) -> torch.Tensor:
    """This is for Forward method implimentation
    """
    return F.linear(
        x,
        self.weight_mu + self.weight_sigma * self.weight_epsilon,
        self.bias_mu + self.bias_sigma * self.bias_epsilon,
    )

@staticmethod
def scale_noise(size: int) -> torch.Tensor:
    """Set scale to make noise . use factorized gaussian noise."""
    x = torch.randn(size)

    return x.sign().mul(x.abs().sqrt())

```

In []:

#network with dueling

In []:

```

class Network(nn.Module):
    def __init__(
        self,
        in_dim: int,
        out_dim: int,
        atom_size: int,
        support: torch.Tensor
    ):
        """Initialization."""
        super(Network, self).__init__()

        self.support = support
        self.out_dim = out_dim
        self.atom_size = atom_size

        # set common feature layer as we do in dueling and duoble DQN
        self.feature_layer = nn.Sequential(
            nn.Linear(in_dim, 128),
            nn.ReLU(),
        )

        # set advantage layer for dueling part
        self.advantage_hidden_layer = NoisyLinear(128, 128)
        self.advantage_layer = NoisyLinear(128, out_dim * atom_size)

        # set value layer for dueing part
        self.value_hidden_layer = NoisyLinear(128, 128)
        self.value_layer = NoisyLinear(128, atom_size)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        """This is forward method implementation."""
        dist = self.dist(x)
        q = torch.sum(dist * self.support, dim=2)

        return q

    def dist(self, x: torch.Tensor) -> torch.Tensor:
        """Get distribution for atoms."""
        feature = self.feature_layer(x)
        adv_hid = F.relu(self.advantage_hidden_layer(feature))

```

```

val_hid = F.relu(self.value_hidden_layer(feature))

advantage = self.advantage_layer(adv_hid).view(
    -1, self.out_dim, self.atom_size
)
value = self.value_layer(val_hid).view(-1, 1, self.atom_size)
q_atoms = value + advantage - advantage.mean(dim=1, keepdim=True)

dist = F.softmax(q_atoms, dim=-1)
dist = dist.clamp(min=1e-3) # for avoiding nans

return dist

def reset_noise(self):
    """Here, I reset all noisy layers."""
    self.advantage_hidden_layer.reset_noise()
    self.advantage_layer.reset_noise()
    self.value_hidden_layer.reset_noise()
    self.value_layer.reset_noise()

```

In []:

```

class DQNAgent3:
    """DQN Agent interacting with environment.

    Attribute:
        env (gym.Env): To read my stock environment
        memory: replay memory to store transitions
        batch_size (int): batch size for sampling
        target_update (int): the update frequency of target q
        gamma: discount rate
        dqn (Network): model to train and select actions
        dqn_target (Network): target model to update
        optimizer (torch.optim): optimizer for training dqn
        transition (list): transition information including state, action, reward, next_state
        support (torch.Tensor): support for categorical dqn
        use_n_step (bool): whether to use n_step memory
        n_step (int): step number to calculate n-step td error
        memory_n (ReplayBuffer): n-step replay buffer, in my case, 3 steps
    """

    def __init__(
        self,
        env: gym.Env,
        memory_size: int,
        batch_size: int,
        target_update: int,
        gamma: float = 0.99,
        # PER parameters
        alpha: float = 0.2,
        beta: float = 0.6,
        prior_eps: float = 1e-6,
        #parameters for categorical DQN. in my modified implimentation, categorical DQN is not
        v_min: float = 0.0,
        v_max: float = 200.0,
        atom_size: int = 51,
        # N-step Learning, I choose 3 steps!!
        n_step: int = 3,
    ):
        """Do intialization!!

        Args:

```

```

env (gym.Env): To read my stock environment
memory: replay memory to store transitions
batch_size (int): batch size for sampling
target_update (int): the update frequency of target q
gamma: discount rate
alpha (float): determines how much prioritization is used
beta (float): determines how much importance sampling is used
prior_eps (float): guarantees every transition can be sampled
n_step (int): step number to calculate n-step td error
"""

obs_dim = env.observation_space.shape[0]
action_dim = env.action_space.n

self.env = env
self.batch_size = batch_size
self.target_update = target_update
self.gamma = gamma
self.atom_size = atom_size
# NoisyNet: All attributes related to epsilon are removed

# device specification as we did in the lab.
self.device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
print(self.device)

# Here, for prioitized experimence memory, I put the PEM parameters.
# memory for 1-step Learning
self.beta = beta
self.prior_eps = prior_eps
self.memory = PrioritizedReplayBuffer(
    obs_dim, memory_size, batch_size, alpha=alpha
)

# memory for N-step Learning, in my case, 3 steps!
self.use_n_step = True if n_step > 1 else False
if self.use_n_step:
    self.n_step = n_step
    self.memory_n = ReplayBuffer(
        obs_dim, memory_size, batch_size, n_step=n_step, gamma=gamma
    )

#DQN parameters
self.v_min = v_min
self.v_max = v_max
self.atom_size = atom_size
self.support = torch.linspace(
    self.v_min, self.v_max, self.atom_size
).to(self.device)

# networks for Rainbow dqn, dqn_target
self.dqn = Network(
    obs_dim, action_dim, self.atom_size, self.support
).to(self.device)
self.dqn_target = Network(
    obs_dim, action_dim, self.atom_size, self.support
).to(self.device)
self.dqn_target.load_state_dict(self.dqn.state_dict())
self.dqn_target.eval()

# optimizer for nural net optimization.

```

```

self.optimizer = optim.Adam(self.dqn.parameters())

# transition to store in memory
# This should be collected !!!!!!!!!!!!!!!!!!!!!!!
self.transition = list()

# mode: train / test
self.is_test = False

def select_action(self, state: np.ndarray) -> np.ndarray:
    """Chose an action from the input state, by following noisy net!"""
    # NoisyNet: by using noisy net, the agent can learn the exploration strategy!!
    selected_action = self.dqn(
        torch.FloatTensor(state).to(self.device)
    ).argmax()
    selected_action = selected_action.detach().cpu().numpy()

    if not self.is_test:
        self.transition = [state, selected_action]

    return selected_action

def step(self, action: np.ndarray) -> Tuple[np.ndarray, np.float64, bool]:
    """Get an action and return the response of my Stock Market Environment!"""
    next_state, reward, done, _ = self.env.step(action)

    if not self.is_test:
        self.transition += [reward, next_state, done]

        # N-step transition, in my case, 3 steps!
        if self.use_n_step:
            one_step_transition = self.memory_n.store(*self.transition)
        # 1-step transition
        else:
            one_step_transition = self.transition

        # add a single step transition
        if one_step_transition:
            self.memory.store(*one_step_transition)

    return next_state, reward, done

def update_model(self) -> torch.Tensor:
    """Update the model by gradient descent."""
    # PER needs beta to calculate weights. This weight leads the better performance!
    samples = self.memory.sample_batch(self.beta)
    weights = torch.FloatTensor(
        samples["weights"].reshape(-1, 1)
    ).to(self.device)
    indices = samples["indices"]

    # 1-step Learning loss. This is left for just in case.
    elementwise_loss = self._compute_dqn_loss(samples, self.gamma)

    # PER: importance sampling before average
    loss = torch.mean(elementwise_loss * weights)

    # N-step Learning loss, in my case 3 steps
    # we are gonna combine 1-step loss and n-step loss so as to
    # prevent high-variance. The original rainbow employs n-step loss only.
    if self.use_n_step:

```



```

        gamma = self.gamma ** self.n_step
        samples = self.memory_n.sample_batch_from_idxxs(indices)
        elementwise_loss_n_loss = self._compute_dqn_loss(samples, gamma)
        elementwise_loss += elementwise_loss_n_loss

        # PER: importance sampling before average
        loss = torch.mean(elementwise_loss * weights)

    self.optimizer.zero_grad()
    loss.backward()
    clip_grad_norm_(self.dqn.parameters(), 10.0)
    self.optimizer.step()

    # PER: update priorities to get the better performance
    loss_for_prior = elementwise_loss.detach().cpu().numpy()
    new_priorities = loss_for_prior + self.prior_eps
    self.memory.update_priorities(indices, new_priorities)

    # NoisyNet: I do reset noise
    self.dqn.reset_noise()
    self.dqn_target.reset_noise()

    return loss.item()

#num_frames = # of episode
def train(self, num_frames: int, num_episode: int):
    """Train the agent."""
    self.is_test = False
    memory_episode=0

    state = self.env.reset()
    for i in range(num_episode):
        update_cnt = 0
        losses = [] #this is collected to show the graph!!
        scores = [] #this is collected to show the graph
        score = 0 #Initial reward

        #this is for loop = # of steps in each episode
        #this should be length of the data!!
        for frame_idx in range(1, num_frames + 1):
            action = self.select_action(state)
            next_state, reward, done = self.step(action)

            state = next_state
            score += reward

        # PER: increase beta
        fraction = min(frame_idx / num_frames, 1.0)
        self.beta = self.beta + fraction * (1.0 - self.beta)

```

```

        # if episode ends
        if done:
            state = self.env.reset()
            scores.append(score)
            score = 0

        # if training is ready
        if len(self.memory) >= self.batch_size:
            loss = self.update_model()
            losses.append(loss)

            update_cnt += 1

        # the update schedule of target q
        if update_cnt % self.target_update == 0:
            self._target_hard_update()

    #this makes the average_values in each episode.
    episode_output = self.env.render()
    memory_episode.append(episode_output)
    profit, max_net_worth, current_step, current_balance, shares_held, total_shares_sold,

    df_episode_output=pd.DataFrame(memory_episode)#
    df_episode_output.to_pickle("./dummy.pkl")

    self.env.close()

    return df_episode_output

def _compute_dqn_loss(self, samples: Dict[str, np.ndarray], gamma: float) -> torch.Tensor
    """Return dqn loss."""
    device = self.device # for shortening the following lines
    state = torch.FloatTensor(samples["obs"]).to(device)
    next_state = torch.FloatTensor(samples["next_obs"]).to(device)
    action = torch.LongTensor(samples["acts"]).to(device)
    reward = torch.FloatTensor(samples["rewards"].reshape(-1, 1)).to(device)
    done = torch.FloatTensor(samples["done"].reshape(-1, 1)).to(device)

    #DQN algorithm
    delta_z = float(self.v_max - self.v_min) / (self.atom_size - 1)

    with torch.no_grad():
        # Double DQN
        next_action = self.dqn(next_state).argmax(1)
        next_dist = self.dqn_target.dist(next_state)
        next_dist = next_dist[range(self.batch_size), next_action]

        t_z = reward + (1 - done) * gamma * self.support
        t_z = t_z.clamp(min=self.v_min, max=self.v_max)
        b = (t_z - self.v_min) / delta_z
        l = b.floor().long()
        u = b.ceil().long()

```

```

        offset = (
            torch.linspace(
                0, (self.batch_size - 1) * self.atom_size, self.batch_size
            ).long()
            .unsqueeze(1)
            .expand(self.batch_size, self.atom_size)
            .to(self.device)
        )

        proj_dist = torch.zeros(next_dist.size(), device=self.device)
        proj_dist.view(-1).index_add_(
            0, (l + offset).view(-1), (next_dist * (u.float() - b)).view(-1)
        )
        proj_dist.view(-1).index_add_(
            0, (u + offset).view(-1), (next_dist * (b - l.float())).view(-1)
        )

        dist = self.dqn.dist(state)
        log_p = torch.log(dist[range(self.batch_size), action])
        elementwise_loss = -(proj_dist * log_p).sum(1)

    return elementwise_loss

def _target_hard_update(self):
    """update the target q"""
    self.dqn_target.load_state_dict(self.dqn.state_dict())

```

experiment11

In []:

```

# parameters
num_frames = 1000
num_episode = 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent3(env, memory_size, batch_size, target_update)
agent.train(num_frames, num_episode)
result11 = pd.read_pickle("./dummy.pkl")
print(result11)

```

experiment12

In []:

```

# parameters
num_frames = 1000
num_episode = 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent3(env, memory_size, batch_size, target_update)

```

```
agent.train(num_frames,num_episode)
result12 = pd.read_pickle("./dummy.pkl")
print(result12)
```

experiment13

In []:

```
# parameters
num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent3(env, memory_size, batch_size, target_update)
agent.train(num_frames,num_episode)
result13 = pd.read_pickle("./dummy.pkl")
print(result13)
```

experiment14

In []:

```
# parameters
num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent3(env, memory_size, batch_size, target_update)
agent.train(num_frames,num_episode)
result14 = pd.read_pickle("./dummy.pkl")
print(result14)
```

experiment 15

In []:

```
# parameters
num_frames = 1000
num_episode= 100
memory_size = 500
batch_size = 64
target_update = 200

# train
agent = DQNAgent3(env, memory_size, batch_size, target_update)
agent.train(num_frames,num_episode)
result15 = pd.read_pickle("./dummy.pkl")
print(result15)
```

Graphs of above experiments

```
In [ ]: avr1=(result1+result2+result3+result4+result5)/5
```

```
In [ ]: plt.plot(result1[0], label = "experiment1")
plt.plot(result2[0], label = "experiment2")
plt.plot(result3[0], label = "experiment3")
plt.plot(result4[0], label = "experiment4")
plt.plot(result5[0], label = "experiment5")
plt.plot(avr1[0], label = "Averaged Profit")
plt.xlabel("Episodes")
plt.ylabel("Profits")
plt.title("Profit through the episodes: Vanilla DQN")
plt.legend()
plt.show()
```

```
In [ ]: avr2=(result6+result7+result8+result9+result10)/5
```

```
In [ ]: plt.plot(result6[0], label = "experiment6")
plt.plot(result7[0], label = "experiment7")
plt.plot(result8[0], label = "experiment8")
plt.plot(result9[0], label = "experiment9")
plt.plot(result10[0], label = "experiment10")
plt.plot(avr2[0], label = "Averaged Profit")
plt.xlabel("Episodes")
plt.ylabel("Profits")
plt.title("Profit through the episodes: Dueling & Double DQN")
plt.legend()
plt.show()
```

```
In [ ]: avr3=(result11+result12+result13+result14+result15)/5
```

```
In [ ]: plt.plot(result11[0], label = "experiment11")
plt.plot(result12[0], label = "experiment12")
plt.plot(result13[0], label = "experiment13")
plt.plot(result14[0], label = "experiment14")
plt.plot(result15[0], label = "experiment15")
plt.plot(avr3[0], label = "Averaged Profit")
plt.xlabel("Episodes")
plt.ylabel("Profits")
plt.title("Profit through the episodes: Rainbow DQN")
plt.legend()
plt.show()
```

```
In [ ]: plt.plot(result1[0], label = "Vanilla: experiment1")
plt.plot(result2[0], label = "Vanilla: experiment2")
plt.plot(result3[0], label = "Vanilla: experiment3")
plt.plot(result4[0], label = "Vanilla: experiment4")
plt.plot(result5[0], label = "Vanilla: experiment5")
plt.plot(result6[0], label = "Dueling & Double: experiment6")
plt.plot(result7[0], label = "Dueling & Double: experiment7")
plt.plot(result8[0], label = "Dueling & Double: experiment8")
plt.plot(result9[0], label = "Dueling & Double: experiment9")
plt.plot(result10[0], label = "Dueling & Double: experiment10")
```

```
plt.plot(result11[0], label = "Rainbow: experiment11")
plt.plot(result12[0], label = "Rainbow: experiment12")
plt.plot(result13[0], label = "Rainbow: experiment13")
plt.plot(result14[0], label = "Rainbow: experiment14")
plt.plot(result15[0], label = "Rainbow: experiment15")
plt.plot(avr1[0], label = "Vanilla: Averaged Profit")
plt.plot(avr2[0], label = "Dueling & Double: Averaged Profit")
plt.plot(avr3[0], label = "Rainbow: Averaged Profit")
plt.xlabel("Episodes")
plt.ylabel("Profits")
plt.title("Profit through the episodes: ALL DQNs")
plt.legend(loc=(1.04,0))
plt.show()
```

In []:

```
plt.plot(avr1[0], label = "Vanilla: Averaged Profit")
plt.plot(avr2[0], label = "Dueling & Double: Averaged Profit")
plt.plot(avr3[0], label = "Rainbow: Averaged Profit")
plt.xlabel("Episodes")
plt.ylabel("Profits")
plt.title("Profit through the episodes: Average DQNs")
plt.legend()
plt.show()
```

In []:

This is the script for task 9 and 10.

Notice: When you run the model first time, Colab always gives you the error: invalid literal for int() with base 10: 'max\n'. In order to resolve this error, go to the utils.py; then, you need to use `docker_limit = int(f.read())` with `docker_limit = 5000000000000000000`

the value of `docker_limit` must be very large. If its value is small, you will get the another error, the lack of memory error. After changing the value, go to Runtime and select restart and run all in order to run the model. After this process, my all codes successfully run.

Another notice: I do not put random seed in my model (because I do not know how to specify the random seed in ray), if you re-run my codes, you may get a bit different result. However, to show my results exactly come from my codes, all the results are described in this script, `Atari_project.ipynb`.

The last notice: my codes only work with GPU. Therefore, before running my code, please change the runtime type on google colab. To specify runtime type, go to the right upper side on the screen and click RAM Disk, and go to view resources; then, choose GPU mode.

```
In [ ]: # Install rllib to use DRL algorithm  
!pip install -U "ray[rllib]" torch  
# this is needed  
!pip install GPUutil
```

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

Download and import Atari games. This is follows by
<https://colab.research.google.com/github/jeffheer/colab-atari-roms>

In order for Atari environment to work on google colab. I choose "SpaceInvaders-v0"

```
In [ ]: ! wget http://www.atarimania.com/roms/Roms.rar
! mkdir /content/ROM/
! unrar e -o+ /content/Roms.rar /content/ROM/
! python -m atari_py.import_roms /content/ROM/
```

From here, I basically follow the lab 09 solution in our class. The website is as follows: <https://moodle.city.ac.uk/course/view.php?id=41658>

```
In [ ]: import ray
import ray.rllib.agents.dqn as dqn

def evaluation_fn(result):
    return result['episode_reward_mean']

def objective_fn(config):
    rewards = []
    trainer = dqn.DQNTrainer(config=config)

    for i in range(20):
        # Perform one iteration of training the policy with DQN
        result = trainer.train()
        intermediate_score = evaluation_fn(result)
        rewards.append(result['episode_reward_mean'])
        # Feed the score back back to Tune.
        tune.report(iterations=i, mean_reward=intermediate_score)
```

In the first time, I specify which version of DQN has the best performance and fix other important hyper parameters explicitly.

```
In [ ]: import ray
```



```

import ray.rllib.agents.dqn as dqn
from ray.tune.logger import pretty_print
from ray import tune

config = dqn.DEFAULT_CONFIG.copy()

#By tuning the improvements of DQN, I specify which DQN type algorithm outperforms in my environment
config["dueling"] = tune.grid_search([True, False])
config["double_q"] = tune.grid_search([True, False])
config['noisy']=tune.grid_search([True, False])
config['prioritized_replay']=tune.grid_search([True, False])
config['n_step']=tune.grid_search([1,5])

#They are explicitly fixed important hyper parameters. They will be tuned on the second hyperparameter search
config["gamma"] = 0.7
config["train_batch_size"] = 32
config['timesteps_per_iteration']=1000
config['target_network_update_freq']=500
config['exploration_config'] =tune.grid_search([{'type': 'EpsilonGreedy', 'initial_epsilon': 1.0, 'epsilon_decay': 0.9999999999999999}])
config["model"] = { "fcnet_hidden": [128, 128, 128],
                    "fcnet_activation": 'relu',
                    }

#This is my chosen Atari environment, Space Invader
config["env"] = "SpaceInvaders-v0"

analysis = tune.run(
    objective_fn,
    metric="mean_reward",
    mode="max",
    num_samples=1,
    config=config)

```

```

In [ ]: print("Best hyperparameters found were: ", analysis.best_config)

```

```

In [ ]: all_dataframes = analysis.trial_dataframes

all_dataframes

```

```

In [ ]:

```

```

In [ ]: df = analysis.results_df

```

```

In [ ]: df[['config.gamma', 'config.double_q', 'config.dueling', 'config.noisy', 'config.prioritized_replay']]

```

```

In [ ]: import matplotlib.pyplot as plt

```

```

In [ ]: # from 0 to 31, 32 combinations!! [0] - [31]
plt.plot(list(all_dataframes.values())[0]['mean_reward'], label= 'Rainbow DQN: double=T, dueling=T, noisy=T')
plt.plot(list(all_dataframes.values())[1]['mean_reward'], label= 'double=F, dueling=T, noisy=T')

```

```

plt.plot(list(all_dataframes.values())[2]['mean_reward'], label= 'double=T, dueling=F, noisy=
plt.plot(list(all_dataframes.values())[3]['mean_reward'], label= 'double=F, dueling=F, noisy=

plt.plot(list(all_dataframes.values())[4]['mean_reward'], label= 'Rainbow DQN: double=T, duel
plt.plot(list(all_dataframes.values())[5]['mean_reward'], label= 'double=F, dueling=T, noisy=
plt.plot(list(all_dataframes.values())[6]['mean_reward'], label= 'double=T, dueling=F, noisy=
plt.plot(list(all_dataframes.values())[7]['mean_reward'], label= 'double=F, dueling=F, noisy=

plt.plot(list(all_dataframes.values())[8]['mean_reward'], label= 'double=T, dueling=T, noisy=
plt.plot(list(all_dataframes.values())[9]['mean_reward'], label= 'double=F, dueling=T, noisy=
plt.plot(list(all_dataframes.values())[10]['mean_reward'], label= 'double=T, dueling=F, noisy
plt.plot(list(all_dataframes.values())[11]['mean_reward'], label= 'PER DQN: double=F, dueling

plt.plot(list(all_dataframes.values())[12]['mean_reward'], label= 'double=T, dueling=T, noisy
plt.plot(list(all_dataframes.values())[13]['mean_reward'], label= 'double=F, dueling=T, noisy
plt.plot(list(all_dataframes.values())[14]['mean_reward'], label= 'double=T, dueling=F, noisy
plt.plot(list(all_dataframes.values())[15]['mean_reward'], label= 'double=F, dueling=F, noisy

plt.plot(list(all_dataframes.values())[16]['mean_reward'], label= 'double=T, dueling=T, noisy
plt.plot(list(all_dataframes.values())[17]['mean_reward'], label= 'double=F, dueling=T, noisy
plt.plot(list(all_dataframes.values())[18]['mean_reward'], label= 'double=T, dueling=F, noisy
plt.plot(list(all_dataframes.values())[19]['mean_reward'], label= 'Noisy DQN: double=F, dueli

plt.plot(list(all_dataframes.values())[20]['mean_reward'], label= 'double=T, dueling=T, noisy
plt.plot(list(all_dataframes.values())[21]['mean_reward'], label= 'double=F, dueling=T, noisy
plt.plot(list(all_dataframes.values())[22]['mean_reward'], label= 'double=T, dueling=F, noisy
plt.plot(list(all_dataframes.values())[23]['mean_reward'], label= 'double=F, dueling=F, noisy

plt.plot(list(all_dataframes.values())[24]['mean_reward'], label= 'double=T, dueling=T, noisy
plt.plot(list(all_dataframes.values())[25]['mean_reward'], label= 'Dueling DQN: double=F, due
plt.plot(list(all_dataframes.values())[26]['mean_reward'], label= 'Double DQN: double=T, duel
plt.plot(list(all_dataframes.values())[27]['mean_reward'], label= 'Vanilla DQN: double=F, due

plt.plot(list(all_dataframes.values())[28]['mean_reward'], label= 'dueling=T, noisy=F, priori
plt.plot(list(all_dataframes.values())[29]['mean_reward'], label= 'BEST DQN: double=F, duelin
plt.plot(list(all_dataframes.values())[30]['mean_reward'], label= 'double=T, dueling=F, noisy
plt.plot(list(all_dataframes.values())[31]['mean_reward'], label= 'N-step DQN: double=F, duel

plt.legend(loc=(1.04,0))
plt.title('Average rewards through epidoes: 32 combinations')
plt.xlabel('Episodes')
plt.ylabel('Average rewards')
plt.show()

```

In []:

```

plt.plot(list(all_dataframes.values())[0]['mean_reward'], label= 'Rainbow DQN: double=T, duel
plt.plot(list(all_dataframes.values())[4]['mean_reward'], label= 'Rainbow DQN: double=T, duel
plt.plot(list(all_dataframes.values())[11]['mean_reward'], label= 'PER DQN: double=F, dueling
plt.plot(list(all_dataframes.values())[15]['mean_reward'], label= 'PER DQN: double=F, dueling
plt.plot(list(all_dataframes.values())[19]['mean_reward'], label= 'Noisy DQN: double=F, dueli
plt.plot(list(all_dataframes.values())[23]['mean_reward'], label= 'Noisy DQN: double=F, dueli
plt.plot(list(all_dataframes.values())[25]['mean_reward'], label= 'Dueling DQN: double=F, due
plt.plot(list(all_dataframes.values())[26]['mean_reward'], label= 'Double DQN: double=T, duel
plt.plot(list(all_dataframes.values())[27]['mean_reward'], label= 'Vanilla DQN: double=F, due
plt.plot(list(all_dataframes.values())[29]['mean_reward'], label= 'BEST DQN: double=F, duelin
plt.plot(list(all_dataframes.values())[30]['mean_reward'], label= 'Double DQN: double=T, duel
plt.plot(list(all_dataframes.values())[31]['mean_reward'], label= 'N-step DQN: double=F, duel

plt.legend(loc=(1.04,0))
plt.title('Average rewards through epidoes: 12 combinations')
plt.xlabel('Episodes')

```

```
plt.ylabel('Average rewards')
plt.show()
```

```
In [ ]:
plt.plot(list(all_dataframes.values())[0]['mean_reward'], label= 'Rainbow DQN: double=T, dueling')
plt.plot(list(all_dataframes.values())[11]['mean_reward'], label= 'PER DQN: double=F, dueling')
plt.plot(list(all_dataframes.values())[19]['mean_reward'], label= 'Noisy DQN: double=F, dueling')
plt.plot(list(all_dataframes.values())[25]['mean_reward'], label= 'Dueling DQN: double=F, dueling')
plt.plot(list(all_dataframes.values())[26]['mean_reward'], label= 'Double DQN: double=T, dueling')
plt.plot(list(all_dataframes.values())[27]['mean_reward'], label= 'Vanilla DQN: double=F, dueling')

plt.legend(loc=(1.04,0))
plt.title('Average rewards through episodes: n_step=1')
plt.xlabel('Episodes')
plt.ylabel('Average rewards')
plt.show()
```

```
In [ ]:
plt.plot(list(all_dataframes.values())[4]['mean_reward'], label= 'Rainbow DQN: double=T, dueling')
plt.plot(list(all_dataframes.values())[15]['mean_reward'], label= 'PER DQN: double=F, dueling')
plt.plot(list(all_dataframes.values())[23]['mean_reward'], label= 'Noisy DQN: double=F, dueling')
plt.plot(list(all_dataframes.values())[29]['mean_reward'], label= 'BEST Dueling DQN: double=F, dueling')
plt.plot(list(all_dataframes.values())[30]['mean_reward'], label= 'Double DQN: double=T, dueling')
plt.plot(list(all_dataframes.values())[31]['mean_reward'], label= 'Vanilla DQN: double=F, dueling')

plt.legend(loc=(1.04,0))
plt.title('Average rewards through episodes: n_step=5')
plt.xlabel('Episodes')
plt.ylabel('Average rewards')
plt.show()
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

```
In [ ]:
```

**noisy F, dueling T, double F, n_step 5,
prioritized F**

with this setting, from now on, I am going to explore the 2nd hyper parameters !

In []:

```
import ray
import ray.rllib.agents.dqn as dqn

def evaluation_fn(result):
    return result['episode_reward_mean']

def objective_fn(config):
    rewards = []
    trainer = dqn.DQNTrainer(config=config)

    for i in range(20):
        # Perform one iteration of training the policy with DQN
        result = trainer.train()
        intermediate_score = evaluation_fn(result)
        rewards.append(result['episode_reward_mean'])
        # Feed the score back back to Tune.
        tune.report(iterations=i, mean_reward=intermediate_score)
```

In []:

```
import ray
import ray.rllib.agents.dqn as dqn
from ray.tune.logger import pretty_print
from ray import tune

config = dqn.DEFAULT_CONFIG.copy()

#These parameters are specified by the first hyper parameter tuning. Thus, a model type, meaning
#which improvements (such as dueling) should be done for SpaceInvader Game for the agent to fire
config["dueling"] = True
config["double_q"] = False
config['noisy']=False
config['prioritized_replay']= False
config['n_step']=5

#They are explicitly fixed important hyper parameters. Now explore different important parameters
config["gamma"] = tune.grid_search([0.2, 0.8])
config["train_batch_size"] = tune.grid_search([32, 64])
config['timesteps_per_iteration']=tune.grid_search([500, 1000])
config['target_network_update_freq']=tune.grid_search([250, 500])

#only the model and policy configurations are kept.
config["model"] = { "fcnet_hiddens": [128, 128, 128],
                    "fcnet_activation": 'relu',
                    }

#if i Change another policy, I got the error, and so I decided to keep using same epsilon greedy policy
config['exploration_config'] = {'type': 'EpsilonGreedy', 'initial_epsilon': 1.0, 'final_epsilon': 0.01}

#This is my chosen Atari environment, Space Invader
config["env"] = "SpaceInvaders-v0"
```

```
analysis = tune.run(
    objective_fn,
    metric="mean_reward",
    mode="max",
    num_samples=1,
    config=config)
```

```
In [ ]: all_dataframes2 = analysis.trial_dataframes

all_dataframes2
```

```
In [ ]: print("Best hyperparameters found were: ", analysis.best_config)
```

```
In [ ]: # from 0 to 15 combinations!! [0] - [15]
plt.plot(list(all_dataframes2.values())[0]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[1]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[2]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[3]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.plot(list(all_dataframes2.values())[4]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[5]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[6]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[7]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.plot(list(all_dataframes2.values())[8]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[9]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[10]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[11]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.plot(list(all_dataframes2.values())[12]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[13]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[14]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[15]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.legend(loc=(1.04,0))
plt.title('Average rewards through episodes: 16 combinations')
plt.xlabel('Episodes')
plt.ylabel('Average rewards')
plt.show()
```

```
In [ ]: plt.plot(list(all_dataframes2.values())[0]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[2]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[4]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[6]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[8]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[10]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[12]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[14]['mean_reward'], label= 'gamma=0.2, target_update=5')

plt.legend(loc=(1.04,0))
plt.title('Average rewards through episodes: gamma=0.2')
plt.xlabel('Episodes')
```

```
plt.ylabel('Average rewards')
plt.show()
```

In []:

```
plt.plot(list(all_dataframes2.values())[1]['mean_reward'], label= 'gamma=0.8, target_update=2')
plt.plot(list(all_dataframes2.values())[3]['mean_reward'], label= 'gamma=0.8, target_update=5')
plt.plot(list(all_dataframes2.values())[5]['mean_reward'], label= 'gamma=0.8, target_update=2')
plt.plot(list(all_dataframes2.values())[7]['mean_reward'], label= 'gamma=0.8, target_update=5')
plt.plot(list(all_dataframes2.values())[9]['mean_reward'], label= 'gamma=0.8, target_update=2')
plt.plot(list(all_dataframes2.values())[11]['mean_reward'], label= 'gamma=0.8, target_update=5')
plt.plot(list(all_dataframes2.values())[13]['mean_reward'], label= 'gamma=0.8, target_update=2')
plt.plot(list(all_dataframes2.values())[15]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.legend(loc=(1.04,0))
plt.title('Average rewards through epidoes: gamma=0.8')
plt.xlabel('Episodes')
plt.ylabel('Average rewards')
plt.show()
```

In []:

```
plt.plot(list(all_dataframes2.values())[0]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[1]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[4]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[5]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[8]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[9]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[12]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[13]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.legend(loc=(1.04,0))
plt.title('Average rewards through epidoes: target_update=250')
plt.xlabel('Episodes')
plt.ylabel('Average rewards')
plt.show()
```

In []:

```
plt.plot(list(all_dataframes2.values())[2]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[3]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.plot(list(all_dataframes2.values())[6]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[7]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.plot(list(all_dataframes2.values())[10]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[11]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.plot(list(all_dataframes2.values())[14]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[15]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.legend(loc=(1.04,0))
plt.title('Average rewards through epidoes: target_update=500')
plt.xlabel('Episodes')
```

```
plt.ylabel('Average rewards')
plt.show()
```

In []:

```
plt.plot(list(all_dataframes2.values())[0]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[1]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[2]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[3]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.plot(list(all_dataframes2.values())[8]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[9]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[10]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[11]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.legend(loc=(1.04,0))
plt.title('Average rewards through epidoes: timestep = 500')
plt.xlabel('Episodes')
plt.ylabel('Average rewards')
plt.show()
```

In []:

```
plt.plot(list(all_dataframes2.values())[4]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[5]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[6]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[7]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.plot(list(all_dataframes2.values())[12]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[13]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[14]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[15]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.legend(loc=(1.04,0))
plt.title('Average rewards through epidoes: timestep = 1000')
plt.xlabel('Episodes')
plt.ylabel('Average rewards')
plt.show()
```

In []:

```
plt.plot(list(all_dataframes2.values())[0]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[1]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[2]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[3]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.plot(list(all_dataframes2.values())[4]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[5]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[6]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[7]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.legend(loc=(1.04,0))
plt.title('Average rewards through epidoes: batch_size=32')
plt.xlabel('Episodes')
plt.ylabel('Average rewards')
plt.show()
```

```
In [ ]: plt.plot(list(all_dataframes2.values())[8]['mean_reward'], label= 'gamma=0.2, target_update=2')
plt.plot(list(all_dataframes2.values())[9]['mean_reward'], label= 'gamma=0.8, target_update=2')

plt.plot(list(all_dataframes2.values())[10]['mean_reward'], label= 'gamma=0.2, target_update=5')
plt.plot(list(all_dataframes2.values())[11]['mean_reward'], label= 'gamma=0.8, target_update=5')

plt.plot(list(all_dataframes2.values())[12]['mean_reward'], label= 'gamma=0.2, target_update=10')
plt.plot(list(all_dataframes2.values())[13]['mean_reward'], label= 'gamma=0.8, target_update=10')

plt.plot(list(all_dataframes2.values())[14]['mean_reward'], label= 'gamma=0.2, target_update=20')
plt.plot(list(all_dataframes2.values())[15]['mean_reward'], label= 'gamma=0.8, target_update=20')

plt.legend(loc=(1.04,0))
plt.title('Average rewards through episodes: batch_size=64')
plt.xlabel('Episodes')
plt.ylabel('Average rewards')
plt.show()
```

```
In [ ]:
```

```
In [ ]:
```

The best model performance with the longer period. Check whether it will converge or not.

I follow the code from lab 07 in this lecture for this convergence task:

<https://moodle.city.ac.uk/course/view.php?id=41658>

```
In [ ]: import ray
import ray.rllib.agents.dqn as dqn
from ray.tune.logger import pretty_print

config = dqn.DEFAULT_CONFIG.copy()

config['num_workers'] = 0
config['num_envs_per_worker'] = 1
config['create_env_on_driver'] = False
config['rollout_fragment_length'] = 4
config['batch_mode'] = 'truncate_episodes'
config['gamma'] = 0.8
config['lr'] = 0.0005
config['train_batch_size'] = 64
config['model'] = {'fcnet_hiddens': [128, 128, 128], 'fcnet_activation': 'relu'}
config['optimizer'] = {}
config['horizon'] = None
config['soft_horizon'] = False
config['no_done_at_end'] = False
config['env'] = 'SpaceInvaders-v0'
config['observation_space'] = None
```



```

config['action_space'] = None
config['env_config'] = {}
config['remote_worker_envs'] = False
config['remote_env_batch_wait_ms'] = 0
config['env_task_fn'] = None
config['render_env'] = False
config['record_env'] = False
config['clip_rewards'] = None
config['normalize_actions'] = True
config['clip_actions'] = False
config['preprocessor_pref'] = 'deepmind'
config['log_level'] = 'WARN'
config['ignore_worker_failures'] = False
config['log_sys_usage'] = True
config['fake_sampler'] = False
config['framework'] = 'tf'
config['eager_tracing'] = False
config['eager_max_retraces'] = 20
config['explore'] = True
config['exploration_config'] = {'type': 'EpsilonGreedy', 'initial_epsilon': 1.0, 'final_epsilon': 0.01}
config['evaluation_interval'] = None
config['evaluation_duration'] = 10
config['evaluation_duration_unit'] = 'episodes'
config['evaluation_parallel_to_training'] = False
config['in_evaluation'] = False
config['evaluation_config'] = {'explore': False}
config['evaluation_num_workers'] = 0
config['custom_eval_function'] = None
config['always_attach_evaluation_results'] = False
config['keep_per_episode_custom_metrics'] = False
config['sample_async'] = False
config['observation_filter'] = 'NoFilter'
config['synchronize_filters'] = True
config['local_tf_session_args'] = {'intra_op_parallelism_threads': 8, 'inter_op_parallelism_threads': 8}
config['compress_observations'] = False
config['metrics_episode_collection_timeout_s'] = 180
config['metrics_num_episodes_for_smoothing'] = 100
config['min_time_s_per_reporting'] = 1
config['min_train_timesteps_per_reporting'] = None
config['min_sample_timesteps_per_reporting'] = None
config['seed'] = None
config['extra_python_environs_for_driver'] = {}
config['extra_python_environs_for_worker'] = {}
config['num_gpus'] = 0
config['_fake_gpus'] = False
config['num_cpus_per_worker'] = 1
config['num_gpus_per_worker'] = 0
config['custom_resources_per_worker'] = {}
config['num_cpus_for_driver'] = 1
config['placement_strategy'] = 'PACK'
config['input'] = 'sampler'
config['input_config'] = {}
config['actions_in_input_normalized'] = False
config['input_evaluation'] = ['is', 'wis']
config['postprocess_inputs'] = False
config['shuffle_buffer_size'] = 0
config['output'] = None
config['output_config'] = {}
config['output_compress_columns'] = ['obs', 'new_obs']
config['output_max_file_size'] = 67108864

```

```

config['multiagent'] = {'policies': {}, 'policy_map_capacity': 100, 'policy_map_cache': None}
config['logger_config'] = None
config['_tf_policy_handles_more_than_one_loss'] = False
config['_disable_preprocessor_api'] = False
config['_disable_action_flattening'] = False
config['_disable_execution_plan_api'] = False
config['disable_env_checking'] = False
config['simple_optimizer'] = -1
config['monitor'] = -1
config['evaluation_num_episodes'] = -1
config['metrics_smoothing_episodes'] = -1
config['timesteps_per_iteration'] = 1000
config['min_iter_time_s'] = -1
config['collect_metrics_timeout'] = -1
config['target_network_update_freq'] = 250
config['target_network_update_freq'] = -1
config['replay_buffer_config'] = {'type': 'MultiAgentReplayBuffer', 'capacity': 50000}
config['store_buffer_in_checkpoints'] = False
config['replay_sequence_length'] = 1
config['lr_schedule'] = None
config['adam_epsilon'] = 1e-08
config['grad_clip'] = 40
config['learning_starts'] = 1000
config['num_atoms'] = 1
config['v_min'] = -10.0
config['v_max'] = 10.0
config['noisy'] = False
config['sigma0'] = 0.5
config['dueling'] = True
config['hiddeens'] = [256] #may lead the error
config['double_q'] = True
config['n_step'] = 5
config['prioritized_replay'] = False
config['prioritized_replay_alpha'] = 0.6
config['prioritized_replay_beta'] = 0.4
config['final_prioritized_replay_beta'] = 0.4
config['prioritized_replay_beta_annealing_timesteps'] = 20000
config['prioritized_replay_eps'] = 1e-06
config['before_learn_on_batch'] = None
config['training_intensinty'] = None
config['worker_side_prioritization'] = False

trainer = dqn.DQNTrainer(config=config)

# Can optionally call trainer.restore(path) to load a checkpoint.

avg_rewards = []

for i in range(200):
    # Perform one iteration of training the policy with PPO
    result = trainer.train()
    #print(pretty_print(result))
    print(result['episode_reward_mean'])
    avg_rewards.append(result['episode_reward_mean'])

    if i % 10 == 0:
        checkpoint = trainer.save()
        print("checkpoint saved at", checkpoint)

```

In []:

```
import matplotlib.pyplot as plt
plt.plot(avg_rewards, 'b.', alpha=.1)
plt.xlabel("Episodes")
plt.ylabel("Episode reward mean")
plt.title("Reward means of the best agent")
plt.show()
```

In []:

Environment for advanced task 7 and 8

In []:

```
#!/usr/bin/env python
# coding: utf-8

# In[1]:

#!/usr/bin/env python
# coding: utf-8

# In[9]:

import random
import json
import gym
from gym import spaces
import pandas as pd
import numpy as np

MAX_ACCOUNT_BALANCE = 2147483647
MAX_NUM_SHARES = 2147483647
MAX_SHARE_PRICE = 5000
MAX_OPEN_POSITIONS = 5
MAX_STEPS = 20000

INITIAL_ACCOUNT_BALANCE = 10000

class StockTradingEnv(gym.Env):
    """A stock trading environment for OpenAI gym"""
    metadata = {'render.modes': ['human']}

    def __init__(self, df):
        super(StockTradingEnv, self).__init__()

        self.df = df
        self.reward_range = (0, MAX_ACCOUNT_BALANCE)
        #self.terminal = False #added from paper code

        # Actions of the format Buy x%, Sell x%, Hold, etc.
        self.action_space = spaces.Discrete(3)

        # Prices contains the OHCL values for the last five prices
        self.observation_space = spaces.Box(
            low=0, high=1, shape=(11,), dtype=np.float16)#Not specofy the range of columns

        #store the data for the graph
        self.asset_memory=[INITIAL_ACCOUNT_BALANCE]
        self.rewards_memory=[]

    def _next_observation(self):
        # Get the stock data points for the last 5 days and scale to between 0-1
        obs = np.array([
```

```

self.df.loc[self.current_step: self.current_step
            , 'Open'].values / MAX_SHARE_PRICE,
self.df.loc[self.current_step: self.current_step
            , 'High'].values / MAX_SHARE_PRICE,
self.df.loc[self.current_step: self.current_step
            , 'Low'].values / MAX_SHARE_PRICE,
self.df.loc[self.current_step: self.current_step
            , 'Close'].values / MAX_SHARE_PRICE,
self.df.loc[self.current_step: self.current_step
            , 'Volume'].values / MAX_NUM_SHARES,
self.balance / MAX_ACCOUNT_BALANCE,
self.shares_held / MAX_NUM_SHARES,
self.cost_basis / MAX_SHARE_PRICE,
self.total_shares_sold / MAX_NUM_SHARES,
self.total_sales_value / (MAX_NUM_SHARES * MAX_SHARE_PRICE),
self.profit / MAX_ACCOUNT_BALANCE
], dtype=np.float16)

return obs

def _take_action(self, action):

    current_price = random.uniform(
        self.df.loc[self.current_step, "Open"], self.df.loc[self.current_step, "Close"])

    action_type = action
    amount = 1 #exchange amount 1 or 0.5 for simplicity and check the convergence

    if action_type < 1:
        # Buy amount % of balance in shares
        total_possible = int(self.balance / current_price)
        shares_bought = int(total_possible * amount)
        prev_cost = self.cost_basis * self.shares_held
        additional_cost = shares_bought * current_price

        self.balance -= additional_cost
        self.cost_basis = (
            prev_cost + additional_cost) / (self.shares_held + shares_bought)
        self.shares_held += shares_bought

    elif action_type < 2:
        # Sell amount % of shares held
        shares_sold = int(self.shares_held * amount)
        self.balance += shares_sold * current_price
        self.shares_held -= shares_sold
        self.total_shares_sold += shares_sold
        self.total_sales_value += shares_sold * current_price

    self.net_worth = self.balance + self.shares_held * current_price

    if self.net_worth > self.max_net_worth:
        self.max_net_worth = self.net_worth

    if self.shares_held == 0:
        self.cost_basis = 0

```

```

def step(self, action):
    # Execute one time step within the environment
    self._take_action(action)

    # df_total_value=pd.DataFrame(self.asset_memory)#added from paper code
    # df_rewards=pd.DataFrame(self.rewards_memory)#added from paper code

    self.current_step += 1

    if self.current_step > len(self.df.loc[:, 'Open'].values) - 6:
        self.current_step = 0

    delay_modifier = (self.current_step / MAX_STEPS)

    obs = self._next_observation()

    self.profit = self.net_worth - INITIAL_ACCOUNT_BALANCE
    max_net_worth = self.max_net_worth
    # self.asset_memory.append(max_net_worth)#added from paper code
    # self.rewards_memory.append(reward)#added from paper code

    done = self.net_worth <= 0.5*INITIAL_ACCOUNT_BALANCE #changed from net_worth to balance
    #meaning that if my networth <my initial half balance, then stop this episode!!

    reward = self.profit * delay_modifier #changed from self.balance to self.profit

    return obs, reward, done, {}

def reset(self):
    # Reset the state of the environment to an initial state
    self.balance = INITIAL_ACCOUNT_BALANCE
    self.net_worth = INITIAL_ACCOUNT_BALANCE
    self.max_net_worth = INITIAL_ACCOUNT_BALANCE
    self.shares_held = 0
    self.cost_basis = 0
    self.total_shares_sold = 0
    self.total_sales_value = 0
    self.profit = 0 # this is added for done and reward by my self.

    # Set the current step to a random point within the data frame
    self.current_step = random.randint(
        0, len(self.df.loc[:, 'Open'].values) - 6)

    return self._next_observation()

```

```

def render(self, mode='human', close=False):
    # Render the environment to the screen
    profit = self.profit #self.net_worth - INITIAL_ACCOUNT_BALANCE
    max_net_worth= self.max_net_worth
    current_step=self.current_step
    current_balance=self.balance
    shares_held = self.shares_held
    total_shares_sold=self.total_shares_sold
    cost_basis=self.cost_basis
    total_sales_value=self.total_sales_value
    net_worth=self.net_worth
    print(f'Step: {self.current_step}')
    print(f'Balance: {self.balance}')
    print(
        f'Shares held: {self.shares_held} (Total sold: {self.total_shares_sold})')
    print(
        f'Avg cost for held shares: {self.cost_basis} (Total sales value: {self.total_sales_value})')
    print(
        f'Net worth: {self.net_worth} (Max net worth: {self.max_net_worth})')
    print(f'Profit: {profit}')

    episode_output=[profit, max_net_worth,current_step,current_balance,shares_held,total_shares_sold]
    return episode_output

```

```
# In[ ]:
```

```
# In[ ]:
```